DEVELOPER DOCUMENTATION

MCProjSim

API REFERENCE

ENDPOINTS · SCHEMAS · EXAMPLES

GET /simulate | POST /validate | POST /generate | MCP Tooling | JSON Export | Typed Models

```
GET   /simulate
POST  /validate
POST  /generate
```

```
{
  "iterations": 10000,
  "confidence": [50, 80, 95]
}
```

JOHAN PERSSON

FIRST EDITION
2026

MCProjSim User Guide
Version `v0.15.1`
Copyright © Johan Persson
Generated on May 5, 2026
Prepared with LaTeX from Markdown sources for duplex printing. Sources for documentation available at
`https://github.com/johan162/mcprojsim/docs`.

API Reference

`mcprojsim`, v0.15.2

Monte Carlo Project Simulation

Johan Persson <johan162@gmail.com>

Generated on May 5, 2026

Contents

1 Introduction - API Reference **5**
1.1 Key Concepts . . . 6
1.2 Root Package . . . 6
1.3 Simulation Workflow . . . 7

2 Core API **9**
2.1 Overview . . . 9
2.2 `SimulationEngine` . . . 10
2.3 `SimulationCancelled` . . . 13
2.4 `SimulationResults` . . . 14
2.5 `CriticalPathRecord` . . . 16
2.6 `TwoPassDelta` . . . 16
2.7 `SprintSimulationEngine` . . . 17
2.8 `SprintPlanningResults` . . . 18

3 Project Models **21**
3.1 Overview . . . 21
3.2 `Project` . . . 22
3.3 `ProjectMetadata` . . . 24
3.4 `Task` . . . 26
3.5 `TaskEstimate` . . . 28
3.6 `Risk` and `RiskImpact` . . . 29
3.7 `ResourceSpec` . . . 30
3.8 `CalendarSpec` . . . 31
3.9 `UncertaintyFactors` . . . 32
3.10 Sprint Planning Models . . . 33
3.10.1 `SprintPlanningSpec` . . . 33

3.10.2 `SprintHistoryEntry` . . . 34
3.10.3 `FutureSprintOverrideSpec` . . . 34
3.10.4 `SprintVolatilitySpec` . . . 35
3.10.5 `SprintSpilloverSpec` . . . 35
3.10.6 `SprintSpilloverBracketSpec` . . . 36
3.10.7 `SprintSicknessSpec` . . . 36
3.11 Enums . . . 37

4 Simulation Results Models **39**
4.1 Overview . . . 39
4.2 `CriticalPathRecord` . . . 40
4.3 `TwoPassDelta` . . . 40
4.4 `SimulationResults` . . . 42

5 Parsers **49**
5.1 Overview . . . 49
5.2 `YAMLParser` . . . 50
5.3 `TOMLParser` . . . 51

6 Configuration **53**
6.1 Overview . . . 53
6.2 `Config` . . . 54
6.3 Config Sub-Models . . . 56
6.3.1 `TShirtSizeConfig` / `StoryPointConfig` . . . 56
6.3.2 `ExperienceProfileConfig` . . . 57
6.3.3 `OutputConfig` . . . 57
6.3.4 `SimulationConfig` . . . 57
6.3.5 `LogNormalConfig` . . . 58
6.3.6 `StaffingConfig` . . . 58
6.3.7 `ConstrainedSchedulingConfig` . . . 58
6.3.8 `SprintDefaultsConfig` . . . 59
6.3.9 `SprintSicknessDefaultsConfig` . . . 60
6.3.10 `CostConfig` . . . 60

7 Exporters **63**
7.1 Overview . . . 63
7.2 `JSONExporter` . . . 64
7.3 `CSVExporter` . . . 65
7.4 `HTMLExporter` . . . 66

8 Analysis Helpers **69**
8.1 Overview . . . 69

8.2 `StatisticalAnalyzer` . . . 70
8.3 `SensitivityAnalyzer` . . . 71
8.4 `CriticalPathAnalyzer` . . . 71
8.5 `StaffingAnalyzer` . . . 72
8.6 `CostAnalyzer` . . . 74

9 Validation Utilities **77**
9.1 Overview . . . 77
9.2 `Validator` . . . 78
9.3 `setup_logging` . . . 78

10 Natural Language Parser **79**
10.1 Overview . . . 79
10.2 `NLProjectParser` . . . 80
10.2.1 Input format . . . 81
10.2.2 Project-level metadata . . . 81
10.2.3 Tasks . . . 82
10.2.4 Resources . . . 84
10.2.5 Calendars . . . 85
10.2.6 Sprint planning . . . 85
10.2.7 Complete example . . . 87
10.3 Data Classes . . . 87
10.3.1 `ParsedProject` . . . 87
10.3.2 `ParsedTask` . . . 88
10.3.3 `ParsedResource` . . . 89
10.3.4 `ParsedRisk` . . . 90
10.3.5 `ParsedCalendar` . . . 90
10.3.6 `ParsedSprintPlanning` . . . 90
10.3.7 `ParsedFutureSprintOverride` . . . 91
10.3.8 `ParsedSprintHistoryEntry` . . . 92

11 API Examples **93**
11.1 Overview . . . 93
11.2 Example 1: Programmatic Simulation with Full Analysis . . . 94
11.3 Example 2: Resource-Constrained and Two-Pass Scheduling . . . 96
11.4 Example 3: Dashboard Integration . . . 98
11.5 Example 4: Sprint Planning with Forecast . . . 100
11.6 Example 5: Batch Processing Multiple Projects . . . 100
11.7 Example 6: Configuration-Driven Customization . . . 102
11.8 Example 7: Progress Callback and Cancellation . . . 104
11.8.1 7a — Progress callback (basic usage) . . . 104
11.8.2 7b — Cancellation from another thread . . . 105

11.8.3 7c — Combining callback with cancellation (GUI pattern) . . 106
11.9 Example 8 — Cost Estimation and Budget Analysis 108

CHAPTER 1

Introduction - API Reference

This document describes the main Python API intended for library consumers. For installation and first steps, see the Quickstart.

The most stable entry points are:

- **Root package imports** from `mcprojsim` — Quick-start for common workflows
- **Data models** from `mcprojsim.models` — Project structure, tasks, risks, estimates
- **File parsers** from `mcprojsim.parsers` — Load YAML/TOML project files
- **Simulation engine** from `mcprojsim` — Run Monte Carlo schedule simulations
- **Exporters** from `mcprojsim.exporters` — Generate JSON, CSV, and HTML reports
- **Configuration** from `mcprojsim.config` — Simulation settings, uncertainty factors, estimate mappings
- **Analysis helpers** from `mcprojsim.analysis` — Statistical, sensitivity, critical-path, and staffing analysis
- **Sprint-planning APIs** from `mcprojsim.planning` and `mcprojsim.models.sprint_simul` — Forecast sprint-based delivery
- **Natural language parser** from `mcprojsim.nl_parser` — Convert text descriptions to project files

!!! note “Internal modules” Modules under `mcprojsim.simulation` (distributions, scheduler, risk_evaluator) and `mcprojsim.utils` are accessible but considered internal. They may change without deprecation notice.

1.1 Key Concepts

Before diving into the API, it helps to understand five concepts that mcprojsim distinguishes:

Elapsed duration vs total effort. *Elapsed duration* is the calendar time from project start to finish — what a Gantt chart shows. *Total effort* is the sum of all person-hours across every task, regardless of parallelism. A 100-hour project done by two people in parallel has ~50 hours elapsed duration but 100 hours of effort. `SimulationResults.mean` and `.percentile()` report elapsed duration; `.total_effort_hours()`, `.effort_percentile()`, and `.effort_durations` report effort.

Dependency-only vs resource-constrained scheduling. When a project defines no resources, the scheduler runs in *dependency-only* mode: tasks start as soon as their predecessors finish, with unlimited parallelism. When resources are defined, *constrained scheduling* activates: tasks compete for finite resource slots, potentially queuing behind other work. Check `results.schedule_mode` and `results.resource_constraints_active` to see which mode was used.

Two-pass scheduling. An optional extension of constrained scheduling. Pass 1 runs a smaller batch of iterations with simple greedy dispatch to rank tasks by criticality index. Pass 2 re-runs the full simulation using those ranks as scheduling priorities. Enable via `SimulationEngine(two_pass=True)` or `config.constrained_scheduling.assignmer = "criticality_two_pass"`. Results include a `two_pass_trace` with pass-1 vs pass-2 comparison data.

Coordination overhead / team size. When `project.project.team_size` is set, mcprojsim applies Brooks's Law–inspired communication overhead: larger teams lose a fraction of capacity to coordination. The staffing analyzer models this via `communication_overhead` and `min_individual_productivity` parameters per experience profile.

Cost estimation. When a project sets `default_hourly_rate` (on project metadata or via config), the engine computes per-iteration costs alongside schedule durations. Cost = (task effort × hourly rate) + fixed costs + risk cost impacts, with an optional overhead multiplier. `SimulationResults` then provides cost percentiles, budget confidence queries, and joint schedule-and-budget analysis. See the SimulationResults cost fields and Example 8.

1.2 Root Package

The root package currently exports:

- `Project`

- `Task`
- `Risk`
- `SimulationEngine`
- `__version__`

```
from mcprojsim import Project, Task, Risk, SimulationEngine, __version__
```

Use these imports when you want the shortest path for common programmatic usage.

1.3 Simulation Workflow

The standard schedule-simulation workflow is:

1. Load a project definition with `YAMLParser` or `TOMLParser`
2. Optionally load a `Config`
3. Run `SimulationEngine`
4. Inspect `SimulationResults`
5. Export the results if needed

```
from mcprojsim import SimulationEngine
from mcprojsim.config import Config
from mcprojsim.parsers import YAMLParser
from mcprojsim.exporters import JSONExporter, HTMLExporter

project = YAMLParser().parse_file("project.yaml")
config = Config.load_from_file("config.yaml")

engine = SimulationEngine(
    iterations=10000,
    random_seed=42,
    config=config,
    show_progress=True,
)
results = engine.run(project)

print(results.mean)
print(results.percentile(90))
print(results.get_critical_path())

# Cost queries (when default_hourly_rate is set on the project)
if results.costs is not None:
    print(results.cost_percentile(90))
    print(results.budget_for_confidence(0.90))

JSONExporter.export(results, "results.json", config=config, project=project)
HTMLExporter.export(results, "results.html", project=project, config=config)
```

Sprint-planning workflow (when `project.sprint_planning.enabled` is true):

1. Load a `Project`
2. Run `SprintSimulationEngine`
3. Inspect `SprintPlanningResults`

```
from mcprojsim.parsers import YAMLParser
from mcprojsim.planning.sprint_engine import SprintSimulationEngine

project = YAMLParser().parse_file("sprint_project.yaml")
engine = SprintSimulationEngine(iterations=5000, random_seed=42)
results = engine.run(project)

print(results.mean)
print(results.percentile(90))
print(results.date_percentile(90))
```

CHAPTER 2

Core API

2.1 Overview

`SimulationEngine` is the central entry point for running Monte Carlo simulations in mcprojsim. It samples task durations (triangular or lognormal), evaluates risks, delegates scheduling to `TaskScheduler`, and aggregates thousands of iterations into a `SimulationResults` object containing duration arrays, percentiles, and critical-path data. The scheduler supports two modes: a fast dependency-only topological sort, and a resource-calendar-constrained mode with optional two-pass criticality prioritisation.

When to use this module: Use it whenever you want to run a simulation programmatically — whether from a parsed YAML project or one built in code.

Capability	Description
Monte Carlo sampling	Runs `iterations` independent samples via triangular or shifted-lognormal distributions
Dependency scheduling	Topological ordering of tasks respecting `depends_on` chains
Resource-constrained scheduling	Assigns named resources with availability, calendars, and experience modifiers
Two-pass criticality mode	Pass 1 builds criticality indices with greedy dispatch; Pass 2 replays with criticality-prioritised assignment

Capability	Description
Risk evaluation	Applies project and task-level risks (percentage or absolute) per iteration
Reproducibility	Flows all randomness through a seeded `numpy.random.RandomState`

Background: Monte Carlo project simulation — Each iteration independently samples every task duration and risk event, then schedules tasks to compute a total elapsed project duration. Repeating this thousands of times yields a distribution from which percentiles (P50, P80, P95, etc.) and critical-path frequencies are derived.

Imports:

```
from mcprojsim import SimulationEngine
from mcprojsim.simulation.engine import SimulationCancelled
from mcprojsim.models.simulation import SimulationResults
```

2.2 `SimulationEngine`

Main entry point for Monte Carlo simulation.

```
from mcprojsim import SimulationEngine

engine = SimulationEngine(
    iterations=10000,
    random_seed=42,
    config=config,
    show_progress=True,
)

results = engine.run(project)
```

Constructor parameters:

Parameter	Type	Default	Description
`iterations`	`int`	`10000`	Number of Monte Carlo iterations to run.
`random_seed`	`int | None`	`None`	Random seed for reproducible sampling.

Parameter	Type	Default	Description
`config`	`Config | None`	`None`	Configuration for uncertainty multipliers, T-shirt mappings, and story-point mappings. Uses `Config.get_default()` when omitted.
`show_progress`	`bool`	`True`	Print progress updates during long runs.
`two_pass`	`bool`	`False`	Enable criticality-two-pass scheduling. Only has effect when resource-constrained scheduling is active. Overrides `config.constrained_scheduling.as`
`pass1_iterations`	`int | None`	`None`	Number of pass-1 iterations for criticality ranking. Overrides `config.constrained_scheduling.pa` when provided. Capped to `iterations`.

Parameter	Type	Default	Description
`progress_callback`	`Callable[[int, int], None] | None`	`None`	Optional callback invoked with `(completed_iterations, total_iterations)` during the run. When provided, stdout progress output is suppressed regardless of the `show_progress` flag. The callback remains active even when `show_progress=False`. Single-pass runs use `iterations` as the total. Two-pass runs report overall work using `pass1_iterations + iterations` as the total, so callback progress remains monotonic across both phases. Parallel runs report as chunks complete, so callback frequency can be higher. See Example 7 for usage.

Parameter	Type	Default	Description
`workers`	`int`	`1`	Number of worker processes for parallel simulation. `1` uses the sequential path (default and recommended for library/MCP callers). Values > 1 distribute iterations across a short-lived `ProcessPoolExecutor` using the `spawn` start method. Parallel dispatch is gated by a workload heuristic (task count, iteration count, two-pass mode, and estimated resource contention). Hard minimum floors still apply at `iterations < 500` or `len(project.tasks) < 5`.

Key methods:

- `run(project: Project) -> SimulationResults` — Run the Monte Carlo simulation and return aggregated results. When two-pass mode is active and resource constraints are present, the engine first runs `pass1_iterations` with greedy scheduling to build criticality indices, then reruns the full simulation with criticality-prioritised dispatch. Raises `SimulationCancelled` if `cancel()` was called before or during the run.
- `cancel() -> None` — Request cancellation of a running simulation. The engine checks an internal flag at the top of each iteration; when set, the current `run()` call raises `SimulationCancelled`. This method is thread-safe: call it from any thread to stop a simulation running in another.

2.3 `SimulationCancelled`

Exception raised when a running simulation is cancelled via `SimulationEngine.cancel()`.

```
from mcprojsim.simulation.engine import SimulationCancelled
```

`SimulationCancelled` is a plain `Exception` subclass with no additional attributes. Catch it to distinguish user-initiated cancellation from other errors:

```
try:
    results = engine.run(project)
except SimulationCancelled:
    print("Simulation was cancelled by the user.")
```

2.4 `SimulationResults`

Holds the output of a simulation run, including durations, summary statistics, percentiles, and critical-path frequency data.

Attributes:

Attribute	Type	Description
`project_name`	`str`	Name of the simulated project.
`iterations`	`int`	Number of iterations used.
`random_seed`	`int | None`	Seed that was used, or `None`.
`hours_per_day`	`float`	Working hours per day (used for day conversions).
`start_date`	`date | None`	Project start date, if provided.
`durations`	`np.ndarray`	Per-iteration elapsed project durations in hours.
`task_durations`	`dict[str, np.ndarray]`	Per-iteration sampled duration arrays keyed by task ID.
`effort_durations`	`np.ndarray`	Per-iteration total person-effort in hours (sum across all tasks).
`mean`	`float`	Mean elapsed duration in hours.
`median`	`float`	Median elapsed duration in hours.
`std_dev`	`float`	Standard deviation of elapsed durations.
`min_duration`	`float`	Minimum elapsed duration observed.
`max_duration`	`float`	Maximum elapsed duration observed.
`skewness`	`float`	Skewness of the duration distribution.
`kurtosis`	`float`	Excess kurtosis of the duration distribution.

Attribute	Type	Description
`percentiles`	`dict[int, float]`	Cached elapsed-duration percentiles (populated on demand).
`effort_percentiles`	`dict[int, float]`	Cached effort percentiles (populated on demand).
`sensitivity`	`dict[str, float]`	Per-task Spearman rank correlations with total elapsed duration.
`task_slack`	`dict[str, float]`	Mean schedule slack per task in hours.
`max_parallel_tasks`	`int`	Peak parallel task count observed across all iterations.
`schedule_mode`	`str`	`"dependency_only"` or `"resource_constrained"`.
`resource_constraints_active`	`bool`	Whether resource-constrained scheduling was used.
`resource_wait_time_hours`	`float`	Mean hours tasks waited for a resource slot.
`resource_utilization`	`float`	Average resource utilization (0.0–1.0).
`calendar_delay_time_hours`	`float`	Mean hours lost to calendar constraints.
`two_pass_trace`	`TwoPassDelta | None`	Pass-1 vs pass-2 comparison payload; `None` unless two-pass mode was active.

Methods:

- `calculate_statistics()` — Compute and cache mean, median, std_dev, min, max, skewness, and kurtosis from the `durations` array.
- `percentile(p: int) -> float` — Elapsed-duration value at percentile p.
- `effort_percentile(p: int) -> float` — Total person-effort value at percentile p.
- `get_critical_path() -> dict[str, float]` — Per-task criticality index (fraction of iterations in which the task was on the critical path).
- `get_critical_path_sequences(top_n: int | None = None) -> list[CriticalPathRecor` — Ordered full critical-path sequences by frequency.
- `get_most_frequent_critical_path() -> CriticalPathRecord | None` — Single most frequent full path sequence.
- `get_histogram_data(bins: int = 50) -> tuple[np.ndarray, np.ndarray]` — Re-

turns `(bin_edges, counts)` for visualisation.

- `probability_of_completion(target_hours: float) -> float` — Fraction of iterations that completed within the given hours.
- `total_effort_hours() -> float` — Sum of per-task mean durations (total person-hours regardless of parallelism).
- `hours_to_working_days(hours: float) -> int` — Convert hours to working days (ceiling).
- `delivery_date(effort_hours: float) -> date | None` — Project delivery date by adding working days to `start_date`; `None` if no start date.
- `get_risk_impact_summary() -> dict[str, dict[str, float]]` — Per-task risk impact statistics (`mean_impact`, `trigger_rate`, `mean_when_triggered`).
- `to_dict() -> dict[str, Any]` — Serialise results to a plain dictionary.

```
print(f"Mean: {results.mean:.2f}")
print(f"Median: {results.median:.2f}")
print(f"P80: {results.percentile(80):.2f}")

criticality = results.get_critical_path()
for task_id, value in criticality.items():
    print(task_id, value)
```

2.5 `CriticalPathRecord`

Aggregated full critical path sequence information.

Fields:

Field	Type	Description
`path`	`tuple[str, ...]`	Ordered task IDs forming the path.
`count`	`int`	Number of iterations this exact path appeared.
`frequency`	`float`	Fraction of total iterations (0.0–1.0).

Methods:

- `format_path() -> str` — returns `"task_a -> task_b -> task_c"`

2.6 `TwoPassDelta`

Traceability payload for two-pass constrained scheduling. Populated in `SimulationResults.two_pa` when `two_pass=True`.

Import path:

```
from mcprojsim.models.simulation import TwoPassDelta
```

Fields:

Field	Type	Description
`enabled`	`bool`	Whether two-pass scheduling was active.
`pass1_iterations`	`int`	Number of iterations run in pass 1.
`pass2_iterations`	`int`	Number of iterations run in pass 2.
`ranking_method`	`str`	Criticality ranking method used (e.g. `"criticality_index"`).
`pass1_mean_hours`	`float`	Pass-1 mean elapsed duration in hours.
`pass1_p50_hours` / `pass1_p80_hours` / `pass1_p90_hours` / `pass1_p95_hours`	`float`	Pass-1 percentile durations.
`pass1_resource_wait_hours`	`float`	Pass-1 mean resource wait time.
`pass1_resource_utilization`	`float`	Pass-1 mean resource utilization.
`pass1_calendar_delay_hours`	`float`	Pass-1 mean calendar delay.
`pass2_*`	`float`	Same fields for pass 2 (full run).
`delta_*`	`float`	Pass-2 minus pass-1 delta (negative = improvement).
`task_criticality_index`	`dict[str, float]`	Per-task criticality index from pass 1.

Methods:

- `to_dict() -> dict[str, Any]` — Serialise the trace payload to a plain dictionary.

2.7 `SprintSimulationEngine`

Entry point for sprint-planning Monte Carlo simulation.

Import path:

```
from mcprojsim.planning.sprint_engine import SprintSimulationEngine
```

Constructor parameters:

Parameter	Type	Default	Description
`iterations`	`int`	`10000`	Number of Monte Carlo iterations.

Parameter	Type	Default	Description
`random_seed`	`int | None`	`None`	Random seed for reproducible sampling.

Key method:

- `run(project: Project) -> SprintPlanningResults` — Run the sprint-planning simulation. Raises `ValueError` if `project.sprint_planning` is not enabled.

2.8 `SprintPlanningResults`

Result model for sprint-planning simulations.

Import path:

```
from mcprojsim.models.sprint_simulation import SprintPlanningResults
```

Attributes:

Attribute	Type	Description
`project_name`	`str`	Name of the simulated project.
`iterations`	`int`	Number of iterations used.
`random_seed`	`int | None`	Seed that was used, or `None`.
`sprint_counts`	`np.ndarray`	Per-iteration sprint-count samples.
`sprint_length_weeks`	`int`	Duration of each sprint in weeks.
`mean`	`float`	Mean total sprint count to completion.
`median`	`float`	Median sprint count.
`std_dev`	`float`	Standard deviation of sprint counts.
`min_sprints`	`float`	Minimum sprint count observed.
`max_sprints`	`float`	Maximum sprint count observed.
`percentiles`	`dict[int, float]`	Sprint count per percentile (populated on demand).
`date_percentiles`	`dict[int, date | None]`	Calendar dates per percentile (populated on demand).
`planned_commitment_guidance`	`float`	Recommended capacity units per sprint.
`historical_diagnostics`	`dict`	Statistics derived from historical velocity data (when provided).
`disruption_statistics`	`dict`	Disruption event statistics.

Attribute	Type	Description
`carryover_statistics`	`dict`	Carryover (incomplete work) statistics.
`spillover_statistics`	`dict`	Task spillover statistics.
`burnup_percentiles`	`list[dict]`	Per-sprint cumulative work by percentile.

Methods:

- `percentile(p: int) -> float` — Total sprint count at percentile p.
- `date_percentile(p: int) -> date | None` — Calendar date for a sprint-count percentile.
- `delivery_date_for_sprints(sprint_count: float) -> date | None` — Convert a sprint count to a projected delivery date.
- `to_dict() -> dict[str, Any]` — Serialise results to a plain dictionary.

Example: Sprint Planning Results

```
from mcprojsim.planning.sprint_engine import SprintSimulationEngine

# Setup
project = YAMLParser().parse_file("sprint_project.yaml")
engine = SprintSimulationEngine(iterations=5000, random_seed=42)
results = engine.run(project)

# Sprint count distribution
print(f"Mean duration: {results.mean:.1f} sprints")
print(f"P50 duration: {results.percentile(50):.1f} sprints")
print(f"P80 duration: {results.percentile(80):.1f} sprints")

# Calendar predictions
print(f"P80 completion date: {results.date_percentile(80)}")

# Commitment guidance
print(f"Recommended sprint capacity: {results.planned_commitment_guidance:.1f} units/sprint"

# Historical diagnostics (if history was provided)
if results.historical_diagnostics:
    hist = results.historical_diagnostics
    print(f"Historical velocity: mean {hist.get('series_statistics', {}).get('completed_unit
    print(f"Historical observations: {hist.get('observation_count', 0)}")
```

CHAPTER 3

Project Models

3.1 Overview

The project model is a set of Pydantic v2 classes that together define every aspect of a simulation input. At the top sits `Project`, which owns `ProjectMetadata`, a list of `Task` objects, optional `Risk` entries, and optional `ResourceSpec`/`CalendarSpec` pools. Pydantic validators enforce structural rules at parse time: dependency integrity (no cycles, no missing IDs), symbolic estimate rules (T-shirt sizes and story points must not include an explicit unit), and threshold validation (red/green confidence thresholds). The YAML and TOML parsers both return a validated `Project` instance — these models are what you read or build when working programmatically.

When to use this module: Use it directly when constructing projects in code, inspecting parsed results, or adding custom validation logic on top of a parsed `Project`.

Capability	Description
Hierarchical schema	`Project` composes metadata, tasks, risks, resources, and calendars in one validated object
Flexible task estimates	`TaskEstimate` supports explicit low/expected/high ranges, T-shirt sizes, or story points
Dependency validation	Model validator rejects unknown `depends_on` IDs and cyclic dependency graphs

Capability	Description
Resource and calendar specs	`ResourceSpec` and `CalendarSpec` define availability, productivity, and working-day rules
Risk modelling	`Risk` entries carry probability, impact type (percentage or absolute), and optional distribution
Config-driven symbolic estimates	T-shirt and story-point mappings are resolved from `Config` at simulation time, not stored on the model

Imports:

```
from mcprojsim.models.project import (
    Project, ProjectMetadata, Task, TaskEstimate, Risk,
    ResourceSpec, CalendarSpec, UncertaintyFactors,
)
```

3.2 `Project`

Represents the complete project definition with metadata, tasks, risks, resources, and calendars.

Fields:

Field	Type	Default	Description
`project`	`ProjectMetadata`	required	Project name, start date, hours/day, and thresholds
`tasks`	`list[Task]`	required	Work items and their dependencies
`project_risks`	`list[Risk]`	`[]`	Top-level project-level risks
`resources`	`list[ResourceSpec]`	`[]`	Named resource pools
`calendars`	`list[CalendarSpec]`	`[]`	Named calendars for holidays/exclusions
`sprint_planning`	`SprintPlanningSpec | None`	`None`	Sprint planning configuration if enabled

Key methods:

- `get_task_by_id(task_id: str) -> Task | None` — Retrieve a task by its ID

Example:

```
from mcprojsim.parsers import YAMLParser

parser = YAMLParser()
project = parser.parse_file("project.yaml")

# Access project metadata
print(f"Project name: {project.project.name}")
print(f"Start date: {project.project.start_date}")
print(f"Total tasks: {len(project.tasks)}")

# Find a specific task
task = project.get_task_by_id("backend_api")
if task:
    print(f"Task: {task.name}, Dependencies: {task.dependencies}")

# Access all risks
for risk in project.project_risks:
    print(f"Risk: {risk.name}, Probability: {risk.probability}")

# Check if sprint planning is configured
if project.sprint_planning and project.sprint_planning.enabled:
    print(f"Sprint length: {project.sprint_planning.sprint_length_weeks} weeks")
```

Example: Building a Project Programmatically

```
from datetime import date
from mcprojsim.models.project import (
    Project, ProjectMetadata, Task, TaskEstimate, Risk,
)

project = Project(
    project=ProjectMetadata(
        name="My API Project",
        start_date=date(2026, 6, 1),
        hours_per_day=8.0,
    ),
    tasks=[
        Task(
            id="design",
            name="API Design",
            estimate=TaskEstimate(low=8, expected=16, high=32),
        ),
        Task(
            id="implement",
            name="Implementation",
            estimate=TaskEstimate(low=40, expected=80, high=160),
            dependencies=["design"],
        ),
        Task(
            id="test",
            name="Testing",
            estimate=TaskEstimate(t_shirt_size="M"),
            dependencies=["implement"],
        ),
    ],
    project_risks=[
        Risk(
            id="scope_creep",
            name="Scope Creep",
            probability=0.3,
            impact=40.0,
        ),
    ],
)
```

3.3 `ProjectMetadata`

Stores top-level project settings and characteristics.

Fields:

Field	Type	Default	Description
`name`	`str`	required	Project name
`description`	`str | None`	`None`	Optional project description
`start_date`	`date`	required	Project start date
`hours_per_day`	`float`	`8.0`	Working hours per day
`currency`	`str | None`	`None`	ISO 4217 currency code for cost tracking (e.g. `"USD"`, `"EUR"`, `"SEK"`). When not set, falls back to `config.cost.currency` (default `"EUR"`)
`default_hourly_rate`	`float | None`	`None`	Default hourly rate applied to all tasks that don't specify a per-task or per-resource rate. Must be ≥ 0.
`overhead_rate`	`float`	`0.0`	Overhead multiplier applied on top of hourly rates (0.0–3.0). E.g. `0.15` adds 15 % overhead.
`secondary_currencies`	`list[str]`	`[]`	Up to 5 additional ISO 4217 currency codes for multi-currency cost output.
`fx_conversion_cost`	`float`	`0.0`	Flat conversion fee fraction (0.0–0.50) applied when converting to secondary currencies.
`fx_overhead_rate`	`float`	`0.0`	Additional overhead fraction for FX conversions (0.0–1.0).
`fx_rates`	`dict[str, float]`	`{}`	Manual FX rate overrides keyed by target currency code. When not set, live rates are fetched.
`confidence_levels`	`list[int]`	`[10, 25, 50, 75, 80, 85, 90, 95, 99]`	Percentiles to include in reports
`probability_red_threshold`	`float`	`0.5`	Probability below which delivery is shown as red

Field	Type	Default	Description
probability_green_threshold	float	0.9	Probability above which delivery is shown as green
distribution	DistributionType	"triangular"	Default task duration distribution
team_size	int | None	None	Total team size; used for coordination overhead and auto-generating resources
t_shirt_size_default_category	str | None	None	Default T-shirt size category for this project (overrides config when set)
uncertainty_factors	UncertaintyFactors | None	None	Project-level uncertainty factor overrides (applied to all tasks unless overridden per-task)

Example:

```
project = parser.parse_file("project.yaml")
meta = project.project

print(f"Project: {meta.name}")
print(f"Start: {meta.start_date}")
print(f"Tracked thresholds: {meta.confidence_levels}")
print(f"Hours/day: {meta.hours_per_day}")
if meta.team_size:
    print(f"Team size: {meta.team_size} people")
```

3.4 Task

Represents a single work item in the project network.

Fields:

Field	Type	Default	Description
id	str	required	Unique task identifier
name	str	required	Human-readable task name
description	str | None	None	Optional task description
estimate	TaskEstimate	required	Duration estimate
dependencies	list[str]	[]	Task IDs this task depends on

Field	Type	Default	Description
uncertainty_factors	UncertaintyFactors | None	UncertaintyFactors()	Uncertainty adjustments; defaults to a new instance with all-medium/colocated levels
resources	list[str]	[]	Required resource names
max_resources	int	1	Maximum number of resources that can be assigned
min_experience_level	int	1	Minimum experience level required (1, 2, or 3)
planning_story_points	int | None	None	Story points override for sprint planning
priority	int | None	None	Scheduling priority hint
spillover_probability_override	float | None	None	Override for sprint spillover probability (0.0–1.0)
risks	list[Risk]	[]	Task-specific risks
fixed_cost	float | None	None	One-time fixed cost for this task (in the project currency). Added to the cost total regardless of duration.

Key methods:

- `has_dependency(task_id: str) -> bool` — Check if this task depends on another task
- `get_planning_story_points() -> int | None` — Return sprint-planning story points (falls back to `estimate.story_points`)

Example:

```
# Iterate over all tasks
for task in project.tasks:
    print(f"Task: {task.id} - {task.name}")
    print(f"  Description: {task.description or 'N/A'}")
    print(f"  Resources: {task.resources}")
    print(f"  Depends on: {task.dependencies}")
    if task.estimate.high is not None:
        print(f"  Estimate: {task.estimate.low}-{task.estimate.expected}-{task.estimate.high} ho
    if task.risks:
        print(f"  Risks: {len(task.risks)} identified")
    print()

# Find tasks with no dependencies
root_tasks = [t for t in project.tasks if not t.dependencies]
print(f"Root tasks (no dependencies): {[t.id for t in root_tasks]}")

# Find the longest task (only among tasks with explicit high estimates)
longest_task = max(project.tasks, key=lambda t: t.estimate.high if t.estimate.high is not None e
print(f"Longest task: {longest_task.id}, up to {longest_task.estimate.high} hours")
```

3.5 `TaskEstimate`

Supports four estimation styles:

- Triangular estimates via `low`, `expected`, and `high`
- Log-normal estimates via `low`, `expected`, and `high`
- T-shirt-sized estimates via `t_shirt_size`
- Story Point estimates via `story_points`

Fields:

Field	Type	Default	Description
`distribution`	`DistributionType | None`	`None`	Override the project-level distribution for this task
`low`	`float | None`	`None`	Minimum duration (optimistic). Also accepted as `min`.
`expected`	`float | None`	`None`	Best-guess duration. Also accepted as `most_likely`.
`high`	`float | None`	`None`	Maximum duration (pessimistic). Also accepted as `max`.
`t_shirt_size`	`str | None`	`None`	Size token, e.g. `"M"`, `"XL"`, or `"epic.L"`

Field	Type	Default	Description
`story_points`	`int | None`	`None`	Story point value (allowed: 1, 2, 3, 5, 8, 13, 21)
`unit`	`EffortUnit | None`	`None` (hours for explicit estimates)	Effort unit (`"hours"`, `"days"`, `"weeks"`). Must not be set for symbolic estimates — the unit comes from configuration.

Example:

```
for task in project.tasks:
    est = task.estimate

    if est.story_points:
        print(f"Task {task.id}: {est.story_points} story points")
    elif est.t_shirt_size:
        print(f"Task {task.id}: {est.t_shirt_size} T-shirt")
    else:
        print(f"Task {task.id}: {est.low}-{est.expected}-{est.high} hours")

    # Show the distribution type (if overridden for this task)
    if est.distribution:
        print(f"  Distribution: {est.distribution.value}")
```

3.6 `Risk` and `RiskImpact`

Represents task-level or project-level risk.

`Risk` fields:

Field	Type	Default	Description
`id`	`str`	required	Unique risk identifier
`name`	`str`	required	Human-readable risk name
`probability`	`float`	required	Probability this risk occurs (0.0–1.0)
`impact`	`float | RiskImpact`	required	Time penalty in hours (float), or a `RiskImpact` object
`description`	`str | None`	`None`	Optional description

Field	Type	Default	Description
`cost_impact`	`float | None`	`None`	Monetary cost impact when this risk triggers (in the project currency). Applied on top of the duration impact.

`RiskImpact` fields:

Field	Type	Default	Description
`type`	`ImpactType`	required	`"percentage"` or `"absolute"`
`value`	`float`	required	Percentage (0–100) or absolute duration
`unit`	`EffortUnit | None`	`None`	Unit for absolute impacts (`"hours"`, `"days"`, `"weeks"`)

Risk methods:

- `get_impact_value(base_duration: float = 0.0, hours_per_day: float = 8.0) -> float` — Calculate numeric impact in hours

Example:

```
# Access project-level risks
for risk in project.project_risks:
    impact = risk.get_impact_value()
    print(f"Project risk: {risk.name}")
    print(f"  Probability: {risk.probability * 100:.0f}%")
    print(f"  Impact: {impact:.1f} hours")

# Access task-level risks
for task in project.tasks:
    for risk in task.risks:
        impact = risk.get_impact_value(base_duration=task.estimate.expected)
        print(f"Task {task.id} risk: {risk.name}")
        print(f"  Probability: {risk.probability * 100:.0f}%")
        print(f"  Impact: {impact:.1f} hours")
```

3.7 `ResourceSpec`

Defines an individual resource (team member) that can be assigned to tasks.

Fields:

Field	Type	Default	Description
`name`	`str | None`	`None`	Resource name (e.g. `"Alice"`). Auto-generated as `resource_001` etc. if omitted.
`id`	`str | None`	`None`	Legacy identifier used as fallback when `name` is not set
`availability`	`float`	`1.0`	Fraction of time available (must be > 0.0, 1.0)
`calendar`	`str`	`"default"`	Calendar identifier to use
`experience_level`	`int`	`2`	Skill level: 1 (junior), 2 (mid), or 3 (senior)
`productivity_level`	`float`	`1.0`	Productivity multiplier (0.1–2.0)
`sickness_prob`	`float`	`0.0`	Probability of absence per scheduling unit (0.0–1.0)
`planned_absence`	`list[date]`	`[]`	Specific dates this resource is unavailable
`hourly_rate`	`float | None`	`None`	Per-resource hourly rate (overrides `ProjectMetadata.default_hourly_rate` for tasks assigned to this resource). Must be 0.

Example:

```
for resource in project.resources:
    print(f"Resource: {resource.name}")
    print(f"  Availability: {resource.availability * 100:.0f}%")
    print(f"  Experience level: {resource.experience_level}")
    print(f"  Calendar: {resource.calendar}")
    if resource.planned_absence:
        print(f"  Planned absence: {len(resource.planned_absence)} days")
```

3.8 `CalendarSpec`

Defines a working calendar for scheduling.

Fields:

Field	Type	Default	Description
`id`	`str`	`"default"`	Calendar identifier
`work_hours_per_day`	`float`	`8.0`	Working hours per day
`work_days`	`list[int]`	`[1, 2, 3, 4, 5]`	Working days of the week (1=Monday … 7=Sunday)
`holidays`	`list[date]`	`[]`	Specific non-working dates

Example:

```
for calendar in project.calendars:
    print(f"Calendar: {calendar.id}")
    print(f"  Work hours/day: {calendar.work_hours_per_day}")
    print(f"  Work days: {calendar.work_days}")
    if calendar.holidays:
        print(f"  Holidays: {len(calendar.holidays)} days")
```

3.9 `UncertaintyFactors`

Applies multipliers to adjust base task duration based on project characteristics.

Fields:

Field	Type	Default	Description
`team_experience`	`str | None`	`"medium"`	Team experience: `"high"`, `"medium"`, or `"low"`
`requirements_maturity`	`str | None`	`"medium"`	Requirements maturity: `"high"`, `"medium"`, or `"low"`
`technical_complexity`	`str | None`	`"medium"`	Technical complexity: `"low"`, `"medium"`, or `"high"`
`team_distribution`	`str | None`	`"colocated"`	Team distribution: `"colocated"` or `"distributed"`
`integration_complexity`	`str | None`	`"medium"`	Integration complexity: `"low"`, `"medium"`, or `"high"`

All fields default to their neutral level (`"medium"` or `"colocated"`), which maps to a 1.0 multiplier and has no effect on the estimate. Only changing a field away from its default affects the task duration. The actual multiplier values are defined in `Config.uncertainty_factors`.

Example:

```
for task in project.tasks:
    if task.uncertainty_factors:
        factors = task.uncertainty_factors
        print(f"Task {task.id} uncertainty adjustments:")
        if factors.team_experience:
            print(f"  Team experience: {factors.team_experience}")
        if factors.technical_complexity:
            print(f"  Technical complexity: {factors.technical_complexity}")
```

3.10 Sprint Planning Models

These models define the sprint-planning input configuration. They are populated from the `sprint_planning:` block in a project YAML file.

3.10.1 `SprintPlanningSpec`

Top-level sprint planning configuration.

Fields:

Field	Type	Default	Description
`enabled`	`bool`	`False`	Whether sprint planning is active
`sprint_length_weeks`	`int`	required	Length of each sprint in weeks
`capacity_mode`	`SprintCapacityMode`	required	`"story_points"` or `"tasks"`
`history`	`list[SprintHistoryEntry]`	`[]`	Historical sprint outcomes (minimum 2 usable rows required when `enabled`)
`planning_confidence_level`	`float`	`0.80`	Confidence level for commitment guidance (0–1)
`removed_work_treatment`	`RemovedWorkTreatment`	`"churn_only"`	How removed scope affects forecasts: `"churn_only"` or `"reduce_backlog"`
`future_sprint_overrides`	`list[FutureSprintOverrideSpec]`	`[]`	Per-sprint capacity adjustments
`volatility_overlay`	`SprintVolatilityOverlaySpec`	`SprintVolatilityOverlaySpec()`	Sprint disruption model
`spillover`	`SprintSpilloverSpec`	`SprintSpilloverSpec()`	Task spillover model
`velocity_model`	`SprintVelocityModel`	`"empirical"`	`"empirical"` or `"neg_binomial"`
`sickness`	`SprintSicknessSpec`	`SprintSicknessSpec()`	Per-person sickness model

3.10.2 `SprintHistoryEntry`

One historical sprint outcome row.

Fields:

Field	Type	Default	Description
`sprint_id`	`str`	required	Unique identifier for this sprint
`sprint_length_weeks`	`int | None`	`None`	Override sprint length (defaults to parent `SprintPlanningSpec.sprint_length_weeks`
`completed_story_points`	`float | None`	`None`	Story points completed (mutually exclusive with `completed_tasks`)
`completed_tasks`	`int | None`	`None`	Tasks completed (mutually exclusive with `completed_story_points`)
`spillover_story_points`	`float`	`0`	Unfinished story points carried over
`spillover_tasks`	`int`	`0`	Unfinished tasks carried over
`added_story_points`	`float`	`0`	Story points added mid-sprint
`added_tasks`	`int`	`0`	Tasks added mid-sprint
`removed_story_points`	`float`	`0`	Story points removed mid-sprint
`removed_tasks`	`int`	`0`	Tasks removed mid-sprint
`holiday_factor`	`float`	`1.0`	Capacity adjustment for holidays
`end_date`	`date | None`	`None`	Sprint end date
`team_size`	`int | None`	`None`	Team size during this sprint
`notes`	`str | None`	`None`	Free-text notes

3.10.3 `FutureSprintOverrideSpec`

Forward-looking capacity adjustment for a specific future sprint. At least one of `sprint_number` or `start_date` must be provided.

Fields:

Field	Type	Default	Description
`sprint_number`	`int | None`	`None`	Target sprint by 1-based number
`start_date`	`date | None`	`None`	Target sprint by start date (must align to a sprint boundary)
`holiday_factor`	`float`	`1.0`	Capacity adjustment for holidays
`capacity_multiplier`	`float`	`1.0`	Overall capacity multiplier for this sprint
`notes`	`str | None`	`None`	Free-text notes

3.10.4 `SprintVolatilitySpec`

Sprint-level disruption overlay (unexpected events reducing capacity).

Fields:

Field	Type	Default	Description
`enabled`	`bool`	`False`	Whether the volatility overlay is active
`disruption_probability`	`float`	`0.0`	Probability of disruption per sprint (0.0–1.0)
`disruption_multiplier_low`	`float`	`1.0`	Low end of triangular capacity-reduction distribution
`disruption_multiplier_expected`	`float`	`1.0`	Expected value of capacity-reduction distribution
`disruption_multiplier_high`	`float`	`1.0`	High end of triangular capacity-reduction distribution

3.10.5 `SprintSpilloverSpec`

Task-level execution spillover model.

Fields:

Field	Type	Default	Description
`enabled`	`bool`	`False`	Whether spillover modeling is active
`model`	`SprintSpilloverModel`	`"table"`	`"table"` or `"logistic"`
`size_reference_points`	`float`	`5.0`	Reference size for normalizing story-point brackets
`size_brackets`	`list[SprintSpilloverBracketSpec]`	(default bracket)	Table-model probability brackets (ascending `max_points`, last entry must be unbounded)
`consumed_fraction_alpha`	`float`	`3.25`	Beta distribution for spilled fraction consumed
`consumed_fraction_beta`	`float`	`1.75`	Beta distribution for spilled fraction consumed
`logistic_slope`	`float`	`1.9`	Logistic model slope parameter
`logistic_intercept`	`float`	`≈ -1.992`	Logistic model intercept parameter

3.10.6 `SprintSpilloverBracketSpec`

One bracket in the table-based spillover model, mapping a story-point range to a spillover probability.

Fields:

Field	Type	Default	Description
`max_points`	`float | None`	`None`	Upper bound of this bracket in story points. `None` creates an unbounded catch-all bracket (must be last).
`probability`	`float`	required	Spillover probability for tasks in this bracket (0.0–1.0)

3.10.7 `SprintSicknessSpec`

Per-person sickness model for sprint capacity.

Fields:

Field	Type	Default	Description
`enabled`	`bool`	`False`	Whether sickness modeling is active
`team_size`	`int | None`	`None`	Override team size for sickness calculations
`probability_per_person_per_week`	`float`	`0.058`	Sickness probability per person per week
`duration_log_mu`	`float`	`0.693`	Log-normal for sickness duration
`duration_log_sigma`	`float`	`0.75`	Log-normal for sickness duration

3.11 Enums

The following enums are part of the model API:

- `DistributionType` — `"triangular"` or `"lognormal"`
- `ImpactType` — `"percentage"` or `"absolute"`
- `EffortUnit` — `"hours"`, `"days"`, or `"weeks"`
- `SprintCapacityMode` — `"story_points"` or `"tasks"`
- `SprintVelocityModel` — `"empirical"` or `"neg_binomial"`
- `SprintSpilloverModel` — `"table"` or `"logistic"`
- `RemovedWorkTreatment` — `"churn_only"` or `"reduce_backlog"`
- `ConstrainedSchedulingAssignmentMode` — `"greedy_single_pass"` or `"criticality_two_pa`

CHAPTER 4

Simulation Results Models

4.1 Overview

`mcprojsim.models.simulation` holds the output models produced by `SimulationEngine` after a Monte Carlo run. `SimulationResults` stores numpy arrays of per-iteration elapsed project durations alongside aggregated statistics, per-task metrics, and critical-path data. A key design distinction is that `durations` represents *elapsed* project time (critical-path length), while `effort_durations` represents *total person-effort* (the sum of all task durations per iteration) — these differ whenever tasks run in parallel.

When to use this module: Access it directly when reading simulation output, computing custom percentiles, querying critical-path sequences, or serialising results to JSON/CSV via `to_dict()`.

Capability	Description
Elapsed duration percentiles	`percentile(p)` queries `durations` (critical-path hours) at any percentile
Total effort percentiles	`effort_percentile(p)` queries `effort_durations` (person-hours) at any percentile
Critical-path task frequency	`get_critical_path()` returns each task's criticality index (0–1) across iterations
Full path sequences	`get_critical_path_sequences()` returns ordered `CriticalPathRecord` objects by frequency

Capability	Description
Delivery date projection	`delivery_date(hours)` adds working days to `start_date`, skipping weekends
Serialisation	`to_dict()` converts the full result to a nested plain dictionary for export

Background: Elapsed duration vs. total effort — Monte Carlo simulation samples each task's duration stochastically. The *elapsed* project duration is the critical-path length (tasks that cannot run in parallel). *Total effort* is the arithmetic sum of all task durations and represents person-hours of work. When tasks run in parallel, elapsed < total effort; keeping both arrays enables accurate both schedule and staffing estimates.

Imports:

```
from mcprojsim.models.simulation import SimulationResults, CriticalPathRecord, TwoPassDelta
```

4.2 `CriticalPathRecord`

Represents one aggregated critical-path sequence across all simulation iterations.

Field	Type	Description
`path`	`tuple[str, ...]`	Ordered sequence of task IDs forming the critical path
`count`	`int`	Number of iterations this exact path was observed
`frequency`	`float`	Fraction of all iterations (0.0–1.0) this path appeared

Method: `format_path() -> str` — Returns the path as a human-readable arrow-separated string (e.g. `"task_a -> task_b -> task_c"`).

4.3 `TwoPassDelta`

Traceability payload produced when the `criticality_two_pass` scheduling mode is active. Stores pass-1 baseline statistics, pass-2 full-run statistics, and the deltas between them.

Pass metadata:

Field	Type	Default	Description
`enabled`	`bool`	`False`	Whether two-pass scheduling was active
`pass1_iterations`	`int`	`0`	Number of pass-1 (baseline) iterations run
`pass2_iterations`	`int`	`0`	Number of pass-2 (priority-ranked) iterations run
`ranking_method`	`str`	`"criticality_index"`	Method used to rank tasks between passes
`task_criticality_index`	`dict[str, float]`	`{}`	Per-task criticality index computed in pass-1

Pass-1 aggregate statistics:

Field	Type	Default
`pass1_mean_hours`	`float`	`0.0`
`pass1_p50_hours`	`float`	`0.0`
`pass1_p80_hours`	`float`	`0.0`
`pass1_p90_hours`	`float`	`0.0`
`pass1_p95_hours`	`float`	`0.0`
`pass1_resource_wait_hours`	`float`	`0.0`
`pass1_resource_utilization`	`float`	`0.0`
`pass1_calendar_delay_hours`	`float`	`0.0`

Pass-2 aggregate statistics (same shape; prefix `pass2_`): `pass2_mean_hours`, `pass2_p50_hours`, `pass2_p80_hours`, `pass2_p90_hours`, `pass2_p95_hours`, `pass2_resource_wait_hours`, `pass2_resource_utilization`, `pass2_calendar_delay_hours`

Deltas (pass-2 minus pass-1; negative = improvement): `delta_mean_hours`, `delta_p50_hours`, `delta_p80_hours`, `delta_p90_hours`, `delta_p95_hours`, `delta_resource_wait_hours`, `delta_resource_utilization`, `delta_calendar_delay_hours`

Method: `to_dict() -> dict[str, Any]` — Serialize the traceability payload to a nested dictionary.

4.4 `SimulationResults`

Holds the complete output of a Monte Carlo simulation run, including all percentiles, critical path analysis, risk summaries, resource diagnostics, and per-task metrics.

Fields:

Field	Type	Default	Description
`project_name`	`str`	required	Name of the project that was simulated
`iterations`	`int`	required	Number of iterations run
`durations`	`np.ndarray`	required	Per-iteration elapsed project duration in hours (the main simulation output array)
`task_durations`	`dict[str, np.ndarray]`	`{}`	Per-task duration arrays (task ID → per-iteration values)
`critical_path_frequency`	`dict[str, int]`	`{}`	Raw count of iterations each task appeared on the critical path
`critical_path_sequences`	`list[CriticalPathRecord]`	`[]`	Full ordered critical-path sequences in descending frequency order
`random_seed`	`int | None`	`None`	Seed used for reproducibility
`probability_red_threshold`	`float`	`0.5`	Probability below which delivery is shown as red
`probability_green_threshold`	`float`	`0.9`	Probability above which delivery is shown as green
`hours_per_day`	`float`	`8.0`	Working hours per calendar day
`start_date`	`date | None`	`None`	Project start date (for delivery date calculations)
`sensitivity`	`dict[str, float]`	`{}`	Per-task Spearman rank correlation with total duration
`task_slack`	`dict[str, float]`	`{}`	Mean schedule slack per task (hours) across all iterations

Field	Type	Default	Description
`max_parallel_tasks`	`int`	`0`	Peak parallel task count observed across all iterations
`schedule_mode`	`str`	`"dependency_only"`	`"dependency_only"` or `"resource_constrained"`
`resource_constrained`	`bool`	`False`	Whether resource-constrained scheduling was used
`resource_wait_time_mean`	`float`	`0.0`	Mean hours tasks waited for a resource slot
`resource_utilization`	`float`	`0.0`	Average resource utilization (0.0–1.0)
`calendar_delay_time_mean`	`float`	`0.0`	Mean hours lost to calendar constraints (weekends, holidays)
`risk_impacts`	`dict[str, np.ndarray]`	`{}`	Per-task risk impact arrays (task ID → per-iteration impact in hours)
`project_risk_impacts`	`np.ndarray`	`[]`	Per-iteration project-level risk impacts in hours
`effort_durations`	`np.ndarray`	`[]`	Per-iteration total person-effort (sum of all task durations); differs from `durations` which is elapsed time
`two_pass_trace`	`TwoPassDelta | None`	`None`	Traceability data when two-pass scheduling was used
`costs`	`np.ndarray | None`	`None`	Per-iteration total project cost. Populated only when cost inputs are present (e.g. `default_hourly_rate`, `fixed_cost`, resource `hourly_rate`).
`task_costs`	`dict[str, np.ndarray] | None`	`None`	Per-task cost arrays (task ID → per-iteration values).
`cost_mean`	`float | None`	`None`	Mean total project cost (populated by `calculate_cost_statistics()`).

Field	Type	Default	Description
`cost_std_dev`	`float | None`	`None`	Standard deviation of total project cost.
`cost_percentiles`	`dict[int, float] | None`	`None`	Cached cost percentiles (populated by `calculate_cost_statistics()`).
`currency`	`str | None`	`None`	ISO 4217 currency code from `ProjectMetadata.currency`.
`cost_analysis`	`CostAnalysis | None`	`None`	Post-processing cost analysis from `CostAnalyzer` (sensitivity and duration correlation). Populated automatically when cost data is available.
`mean`	`float`	`0.0`	Mean elapsed project duration (hours)
`median`	`float`	`0.0`	Median elapsed project duration (hours)
`std_dev`	`float`	`0.0`	Standard deviation of elapsed duration
`min_duration`	`float`	`0.0`	Minimum observed elapsed duration
`max_duration`	`float`	`0.0`	Maximum observed elapsed duration
`skewness`	`float`	`0.0`	Skewness of the duration distribution
`kurtosis`	`float`	`0.0`	Excess kurtosis of the duration distribution
`percentiles`	`dict[int, float]`	`{}`	Pre-computed elapsed duration percentiles (hours)
`effort_percentiles`	`dict[int, float]`	`{}`	Pre-computed total effort percentiles (person-hours)

Elapsed duration vs. total effort: `durations` and `mean`/`percentiles` represent the *elapsed* project timeline (critical-path time, accounting for parallelism). `effort_durations` and `effort_percentiles` represent the total *person-hours* of work across all tasks — this will always be the elapsed duration when tasks run in parallel.

Key methods:

- **`calculate_statistics() -> None`** — Populate `mean`, `median`, `std_dev`, `min_duration`, `max_duration`, `skewness`, and `kurtosis` from `durations`. Called automatically by `SimulationEngine` after the run.
- **`percentile(p: int) -> float`** — Get elapsed duration for a specific percentile
- **`effort_percentile(p: int) -> float`** — Get total effort for a specific percentile (falls back to `total_effort_hours()` when per-iteration effort data is unavailable)
- **`probability_of_completion(target_hours: float) -> float`** — Calculate probability of finishing within a target duration
- **`hours_to_working_days(hours: float) -> int`** — Convert hours to working days (ceiling rounding)
- **`delivery_date(effort_hours: float) -> date | None`** — Convert elapsed duration to a calendar date (skips weekends; returns `None` if `start_date` is unset)
- **`get_critical_path() -> dict[str, float]`** — Per-task criticality index (0.0–1.0, frequency on critical path), derived from `critical_path_frequency`
- **`get_critical_path_sequences(top_n: int | None = None) -> list[CriticalPathRecor`** — Most frequent full paths (up to `top_n`)
- **`get_most_frequent_critical_path() -> CriticalPathRecord | None`** — Single most common critical path
- **`get_histogram_data(bins: int = 50) -> tuple[np.ndarray, np.ndarray]`** — Bin edges and counts for distribution visualization
- **`get_risk_impact_summary() -> dict[str, dict[str, float]]`** — Per-task risk triggering and impact statistics (`mean_impact`, `trigger_rate`, `mean_when_triggered`)
- **`total_effort_hours() -> float`** — Sum of per-task mean durations (total person-hours)
- **`to_dict() -> dict[str, Any]`** — Serialize results to a dictionary

Cost methods (available only when cost data is populated — i.e. when any cost input is present in the project):

- **`calculate_cost_statistics() -> None`** — Populate `cost_mean`, `cost_std_dev`, and `cost_percentiles` from the `costs` array. Called automatically by `SimulationEngine` after the run.
- **`cost_percentile(p: int) -> float`** — Cost value at percentile p.
- **`probability_within_budget(target_budget: float) -> float`** — Fraction of iterations where total cost was `target_budget`.
- **`budget_confidence_interval(target_budget: float, confidence_level: float = 0.95) -> tuple[float, float, float]`** — Returns `(point_estimate, lower_bound, upper_bound)` where `point_estimate` is the fraction of iterations within budget and the bounds form the confidence interval for that probability.
- **`budget_for_confidence(confidence: float) -> float`** — The budget amount

needed to achieve a given confidence level (e.g. `0.80` for 80 % certainty of staying within budget).

- **`joint_probability(target_hours: float, target_budget: float) -> float`** — Probability that the project completes within *both* the target duration and the target budget simultaneously.

Example: Complete Results Query

```
from mcprojsim import SimulationEngine
from mcprojsim.config import Config
from mcprojsim.parsers import YAMLParser

# Setup
project = YAMLParser().parse_file("project.yaml")
config = Config.get_default()
engine = SimulationEngine(iterations=10000, random_seed=42, config=config)
results = engine.run(project)

# Query duration statistics
print(f"Mean duration: {results.mean:.1f} hours ({results.mean / results.hours_per_day:.1f}
print(f"Median (P50): {results.percentile(50):.1f} hours")
print(f"P80 estimate: {results.percentile(80):.1f} hours")
print(f"P95 estimate: {results.percentile(95):.1f} hours")

# Calculate success odds for a deadline
deadline_hours = 500
success_prob = results.probability_of_completion(deadline_hours)
print(f"\nProbability of completion within {deadline_hours} hours: {success_prob*100:.1f}%")

# Get a delivery date for a specific duration
delivery = results.delivery_date(results.percentile(80))
print(f"P80 delivery date: {delivery}")

# Analyze critical path
critical_tasks = results.get_critical_path()
top_critical = sorted(critical_tasks.items(), key=lambda x: x[1], reverse=True)[:5]
print("\nTop 5 critical tasks (frequency on critical path):")
for task_id, criticality in top_critical:
    print(f"  {task_id}: {criticality*100:.1f}%")

# Get most common path
most_common = results.get_most_frequent_critical_path()
if most_common:
    print(f"\nMost frequent path ({most_common.frequency*100:.1f}%): {most_common.format_pat

# Show histogram data (for charting)
bin_edges, counts = results.get_histogram_data(bins=40)
print(f"\nHistogram: {len(bin_edges)-1} bins, total observations: {sum(counts)}")

# Risk analysis
risk_summary = results.get_risk_impact_summary()
for task_id, stats in risk_summary.items():
    if stats['trigger_rate'] > 0.05:  # Show risks triggered in >5% of iterations
        print(f"Task {task_id}: {stats['trigger_rate']*100:.1f}% trigger rate, "
              f"mean impact {stats['mean_impact']:.1f}h")

# Resource constraints info (if applicable)
if results.resource_constraints_active:
    print(f"\nResource utilization: {results.resource_utilization*100:.1f}%")
```

Example: Querying Cost Results

Cost tracking activates automatically when any cost input is present in the project (`default_hourly_rate`, `Task.fixed_cost`, `ResourceSpec.hourly_rate`, or `Risk.cost_impact`).

```
if results.costs is not None:
    # Cost percentiles
    print(f"Mean cost: {results.currency} {results.cost_mean:,.0f}")
    print(f"P50 cost: {results.currency} {results.cost_percentile(50):,.0f}")
    print(f"P80 cost: {results.currency} {results.cost_percentile(80):,.0f}")
    print(f"P95 cost: {results.currency} {results.cost_percentile(95):,.0f}")

    # Budget analysis
    budget = 250_000
    prob = results.probability_within_budget(budget)
    print(f"\nProbability within {results.currency} {budget:,}: {prob*100:.1f}%")

    # What budget gives 80% confidence?
    budget_80 = results.budget_for_confidence(0.80)
    print(f"Budget for 80% confidence: {results.currency} {budget_80:,.0f}")

    # Joint probability: finish within both time AND budget
    jp = results.joint_probability(target_hours=500, target_budget=budget)
    print(f"P(within 500h AND within {results.currency} {budget:,}): {jp*100:.1f}%")

    # Cost sensitivity (which tasks drive cost variance)
    if results.cost_analysis:
        for task_id, corr in sorted(
            results.cost_analysis.sensitivity.items(),
            key=lambda x: abs(x[1]),
            reverse=True,
        )[:5]:
            print(f"  {task_id}: cost sensitivity = {corr:.3f}")
```

CHAPTER 5

Parsers

5.1 Overview

The `mcprojsim.parsers` module translates YAML and TOML project definition files into validated `Project` model objects. `YAMLParser` and `TOMLParser` share an identical public interface — `parse_file`, `parse_dict`, and `validate_file` — so they are interchangeable given a file format. Both parsers use `error_reporting.py` to preserve file-and-line context when surfacing validation errors, so error messages point to the exact source location rather than generic Pydantic output.

When to use this module: Use the parsers when loading a project from disk in a script or tool integration; use `validate_file` for non-raising pre-flight checks before a simulation run.

Capability	Description
YAML loading	`YAMLParser.parse_file` reads and parses a `.yaml` project file into a `Project`
TOML loading	`TOMLParser.parse_file` reads and parses a `.toml` project file into a `Project`
In-memory parsing	`parse_dict` constructs a `Project` from an already-loaded dictionary
Non-raising validation	`validate_file` returns `(bool, error_message)` without raising an exception

Capability	Description
Line-aware errors	All errors include the file path and source line number via `error_reporting.py`

Imports:

```
from mcprojsim.parsers import YAMLParser, TOMLParser
```

5.2 `YAMLParser`

Parses YAML project files into `Project` objects.

Method	Signature	Description
`parse_file`	`(file_path: Path | str) -> Project`	Load and parse a YAML file. Raises `FileNotFoundError` if the file is missing, `ValueError` on invalid content.
`parse_dict`	`(data: dict[str, Any]) -> Project`	Parse project data from a dictionary. Raises `ValueError` on invalid data.
`validate_file`	`(file_path: Path | str) -> tuple[bool, str]`	Validate without constructing a `Project`. Returns `(True, "")` on success or `(False, error_message)` on failure.

```
from mcprojsim.parsers import YAMLParser

parser = YAMLParser()

# Parse directly
project = parser.parse_file("project.yaml")

# Validate without raising
is_valid, error = parser.validate_file("project.yaml")
if not is_valid:
    print(f"Validation error: {error}")
```

5.3 `TOMLParser`

Identical interface to `YAMLParser`, but for TOML project files.

Method	Signature	Description
`parse_file`	`(file_path: Path | str) -> Project`	Load and parse a TOML file. Raises `FileNotFoundError` if the file is missing, `ValueError` on invalid content.
`parse_dict`	`(data: dict[str, Any]) -> Project`	Parse project data from a dictionary. Raises `ValueError` on invalid data.
`validate_file`	`(file_path: Path | str) -> tuple[bool, str]`	Validate without constructing a `Project`. Returns `(True, "")` on success or `(False, error_message)` on failure.

```
from mcprojsim.parsers import TOMLParser

parser = TOMLParser()
project = parser.parse_file("project.toml")
is_valid, error = parser.validate_file("project.toml")
```

CHAPTER 6

Configuration

6.1 Overview

`Config` is the single source of truth for all default values in mcprojsim — uncertainty multipliers, T-shirt-size and story-point mappings, output settings, staffing profiles, constrained-scheduling parameters, and sprint-planning defaults. `Config.load_from_file()` merges YAML overrides onto the built-in defaults rather than replacing the whole structure, so you only need to specify the fields you want to change. Use this module whenever you need to customise simulation behaviour programmatically or inspect the defaults your project file inherits.

When to use this module: Customise T-shirt sizes, story-point ranges, uncertainty factors, constrained-scheduling assignment mode, or output format settings without modifying the project YAML.

Capability	Description
T-shirt size mappings	Configurable low/expected/high hour ranges per size label, organised by category
Story-point mappings	Configurable low/expected/high day ranges per point value
Uncertainty factor multipliers	Per-factor, per-level multipliers (e.g. `team_experience: high → 0.90`) applied during simulation

Capability	Description
Sub-config classes	`SimulationConfig`, `LognormalConfig`, `OutputConfig`, `StaffingConfig`, `ConstrainedSchedulingConfig`, `SprintDefaultsConfig`
Merge-onto-defaults loading	`Config.load_from_file()` deep-merges a YAML file onto built-in defaults
Programmatic defaults	`Config.get_default()` returns a fully populated instance with no file required

Imports:

```
from mcprojsim.config import Config
```

6.2 `Config`

Application-wide configuration for simulation settings, output formatting, uncertainty-factor multipliers, T-shirt-size mappings, story-point mappings, staffing recommendations, and sprint-planning defaults.

Top-level fields:

Field	Type	Description
`uncertainty_factors`	`dict[str, dict[str, float]]`	Multipliers for uncertainty factors
`t_shirt_sizes`	`dict[str, dict[str, TShirtSizeConfig]]`	T-shirt size mappings by category
`t_shirt_size_default_category`		Default category for bare T-shirt sizes (default: `"epic"`)
`t_shirt_size_unit`	`EffortUnit`	Unit for T-shirt size estimates (default: `HOURS`)
`story_points`	`dict[int, StoryPointConfig]`	Story point mappings
`story_point_unit`	`EffortUnit`	Unit for story point estimates (default: `DAYS`)
`simulation`	`SimulationConfig`	Simulation defaults
`lognormal`	`LogNormalConfig`	Log-normal distribution settings
`output`	`OutputConfig`	Output and export settings

Field	Type	Description
`staffing`	`StaffingConfig`	Staffing analysis configuration
`constrained_scheduling`	`ConstrainedSchedulingConfig`	Resource-constrained scheduling settings
`sprint_defaults`	`SprintDefaultsConfig`	Sprint planning defaults
`cost`	`CostConfig`	Cost estimation defaults

Key methods:

Method	Signature	Description
`Config.load_from_file`	`(config_path: Path | str) -> Config`	Load configuration from a YAML file, merging overrides onto built-in defaults.
`Config.get_default`	`() -> Config`	Return a `Config` instance populated with built-in defaults.
`get_uncertainty_multiplier`	`(factor_name: str, level: str) -> float`	Look up a multiplier for a named uncertainty factor and level.
`get_t_shirt_size`	`(size: str) -> TShirtSizeConfig | None`	Resolve a T-shirt size label. Returns `None` on unrecognised input.
`resolve_t_shirt_size`	`(size: str) -> TShirtSizeConfig`	Resolve a T-shirt size label. Raises `ValueError` on unrecognised input.
`get_t_shirt_categories`	`() -> list[str]`	List all configured T-shirt size categories.
`get_story_point`	`(points: int) -> StoryPointConfig | None`	Resolve a story point value. Returns `None` if not configured.
`get_lognormal_high_z_value`	`() -> float`	Return the Z-score corresponding to `lognormal.high_percentile`.

Example: Working with Configuration

```
from mcprojsim.config import Config

# Load custom config
config = Config.load_from_file("config.yaml")

# Or use defaults
config = Config.get_default()

# Query uncertainty factors
team_multiplier = config.get_uncertainty_multiplier("team_experience", "high")
print(f"High-experience multiplier: {team_multiplier}")

# Query T-shirt sizes
size_config = config.get_t_shirt_size("M")
if size_config:
    print(f"M estimate: {size_config.low}-{size_config.expected}-{size_config.high} hours")

# Query story points
sp = config.get_story_point(5)
if sp:
    print(f"5 story points: {sp.low}-{sp.expected}-{sp.high} hours")

# List T-shirt categories
categories = config.get_t_shirt_categories()
print(f"Available categories: {', '.join(categories)}")

# Access output settings
print(f"Default histogram bins: {config.output.number_bins}")
print(f"Critical path limit: {config.output.critical_path_report_limit}")
```

6.3 Config Sub-Models

6.3.1 `TShirtSizeConfig` / `StoryPointConfig`

Both inherit from `EstimateRangeConfig` and contain:

Field	Type	Description
`low`	`float`	Optimistic estimate
`expected`	`float`	Best estimate
`high`	`float`	Pessimistic estimate

Returned by `config.get_t_shirt_size()` and `config.get_story_point()`.

6.3.2 `ExperienceProfileConfig`

Productivity and overhead parameters for an experience profile.

Field	Type	Default	Description
`productivity_factor`	`float`	`1.0`	Effective output multiplier
`communication_overhead`	`float`	`0.06`	Fraction of capacity lost to coordination (0.0–1.0)

Used inside `StaffingConfig.experience_profiles`.

6.3.3 `OutputConfig`

Settings for simulation output and export behavior.

Field	Type	Default	Description
`formats`	`list[str]`	`["json", "csv", "html"]`	Default export formats
`include_histogram_data`	`bool`	`True`	Include histogram data in exports
`number_bins`	`int`	`50`	Number of bins for histogram charts
`critical_path_report_limit`	`int`	`2`	Max critical path sequences to show

Histogram binning note: The `number_bins` setting is used by all exporters when generating distribution charts in JSON, CSV, and HTML reports. You can also override this per-run via the `--number-bins` CLI flag.

6.3.4 `SimulationConfig`

Defaults for simulation runs.

Field	Type	Default	Description
`default_iterations`	`int`	`10000`	Default Monte Carlo iteration count
`random_seed`	`int | None`	`None`	Default random seed
`max_stored_critical_paths`	`int`	`20`	Maximum distinct critical paths to track

6.3.5 `LogNormalConfig`

Shifted log-normal interpretation settings.

Field	Type	Default	Description
`high_percentile`	`int`	`95`	The percentile the "high" estimate maps to. Allowed values: `70`, `75`, `80`, `85`, `90`, `95`, `99`.

6.3.6 `StaffingConfig`

Configuration for staffing recommendations.

Field	Type	Default	Description
`effort_percentile`	`int | None`	`None`	Percentile to base staffing on (e.g., `80` for P80). When `None`, mean effort is used.
`min_individual_productivity`	`float`	`0.25`	Floor for individual productivity after communication overhead.
`experience_profiles`	`dict[str, ExperienceProfileConfig]`	`"senior", "mixed", "junior"`	Named team experience profiles.

Example:

```
# Use P80 effort for staffing (more conservative)
config = Config.get_default()
config.staffing.effort_percentile = 80

engine = SimulationEngine(iterations=10000, config=config)
results = engine.run(project)
```

6.3.7 `ConstrainedSchedulingConfig`

Settings for resource-constrained scheduling.

Field	Type	Default	Description
`sickness_prob`	`float`	`0.0`	Default per-resource sickness probability when not specified on the resource.
`assignment_mode`	`ConstrainedSchedulingAssignmentMode`	`"greedy_single_pass"`	Scheduling dispatch policy. `"greedy_single_pass"` uses deterministic ID-order greedy dispatch; `"criticality_two_pass"` runs a criticality-ranking pass first.
`pass1_iterations`	`int`	`1000`	Iteration count for the criticality-ranking pass when using `"criticality_two_pass"`.

6.3.8 `SprintDefaultsConfig`

Company-wide defaults for sprint-planning behavior.

Field	Type	Default	Description
`planning_confidence_level`	`float`	`0.80`	Default confidence for commitment guidance.
`removed_work_treatment`	`"churn_only"` | `"reduce_backlog"`	`"churn_only"`	How removed work is accounted for in velocity.
`velocity_model`	`"empirical"` | `"neg_binomial"`	`"empirical"`	Velocity sampling model.
`volatility_disruption_probability`	`float`	`0.0`	Default disruption probability per sprint.
`volatility_disruption_multiplier_low`	`float`	`1.0`	Low-end disruption multiplier.
`volatility_disruption_multiplier_expected`	`float`	`1.0`	Expected disruption multiplier.
`volatility_disruption_multiplier_high`	`float`	`1.0`	High-end disruption multiplier.
`spillover_model`	`"table"` | `"logistic"`	`"table"`	Spillover estimation model.

Field	Type	Default	Description
`spillover_size_reference_points`	`float`	`5.0`	Reference sprint size for the spillover table.
`spillover_size_brackets`	`list[dict[str, float | None]]`	See defaults	Bracket definitions for the table-based spillover model.
`spillover_consumed_fraction_alpha`	`float`	`3.25`	Beta distribution alpha for consumed-fraction sampling.
`spillover_consumed_fraction_beta`	`float`	`1.75`	Beta distribution beta for consumed-fraction sampling.
`spillover_logistic_slope`	`float`	`1.9`	Slope for the logistic spillover model.
`spillover_logistic_intercept`	`float`	`~-1.99`	Intercept for the logistic spillover model.
`sickness`	`SprintSicknessDefaultsConfig`	See below	Company-wide sprint sickness defaults.

6.3.9 `SprintSicknessDefaultsConfig`

Company-wide defaults for sprint sickness modelling. Nested inside `SprintDefaultsConfig.sickne`

Field	Type	Default	Description
`enabled`	`bool`	`False`	Whether sickness is modelled by default in sprint simulations.
`probability_per_person_per_week`	`float`	`0.058`	Probability that a team member falls sick in any given week.
`duration_log_mu`	`float`	`0.693`	Log-mean of the log-normal sickness duration distribution.
`duration_log_sigma`	`float`	`0.75`	Log-standard-deviation of the sickness duration distribution.

6.3.10 `CostConfig`

Defaults for cost estimation. Nested inside `Config.cost`.

Field	Type	Default	Description
`default_hourly_rate`	`float | None`	`None`	Default hourly rate when not specified on the project or resource.
`overhead_rate`	`float`	`0.0`	Default overhead multiplier (0.0–3.0).
`currency`	`str`	`"EUR"`	Default currency code (ISO 4217).
`include_in_output`	`bool`	`True`	Whether cost data is included in export output.

Example:

```
config = Config.get_default()
print(f"Default cost currency: {config.cost.currency}")
print(f"Default overhead rate: {config.cost.overhead_rate}")
print(f"Include cost in output: {config.cost.include_in_output}")
```

CHAPTER 7

Exporters

7.1 Overview

The exporters module converts a `SimulationResults` object into persistent output files. All three exporters — `JSONExporter`, `CSVExporter`, and `HTMLExporter` — expose the same design: a single static `export()` class method that writes directly to a file path. Choose JSON for downstream tooling and API integration, CSV for spreadsheet analysis, and HTML for human-readable reports that can be shared without any tooling. Sprint-planning results and the original `Project` definition are optional enrichments accepted by all three. The `include_historic_base` flag embeds baseline comparison data and is supported only by JSON and HTML; passing it to CSV raises a `TypeError` (unexpected keyword argument).

When to use this module: Write simulation output to disk from your own Python code, or extend a pipeline that consumes JSON, CSV, or HTML artefacts.

Capability	Description
JSON export	Full results: percentiles, statistics, histogram, critical paths, risk summaries, sprint data
CSV export	Flat table: metrics, percentiles, critical paths, histogram bins, risk impact, resource diagnostics
HTML export	Self-contained visual report with embedded charts (requires `matplotlib`)
Sprint results embedding	All three exporters accept optional `SprintPlanningResults`

Capability	Description
Historic baseline	`include_historic_base=True` embeds baseline comparison data (JSON and HTML only)
Config-driven output	Histogram bin count and critical-path report limit come from `Config.output`

Imports:

```
from mcprojsim.exporters import JSONExporter, CSVExporter, HTMLExporter
```

All exporters support histogram data, critical-path output, sprint-planning results, and historic-base data (when available). The histogram bin count defaults to 50 but is controlled by `config.output.number_bins`.

7.2 `JSONExporter`

Exports comprehensive results including all percentiles, statistics, histogram data, critical paths, risk summaries, and sprint-planning data to JSON.

Method: `export(results, output_path, config=None, critical_path_limit=None, sprint_results=None, project=None, include_historic_base=False, full_cost_detail=False, fx_provider=None, target_budget=None, target_hours=None) -> None`

Parameter	Type	Default	Description
`results`	`SimulationResults`	required	Simulation results object.
`output_path`	`Path | str`	required	File path for JSON output.
`config`	`Config | None`	`None`	Active configuration; controls histogram bin count and other settings.
`critical_path_limit`	`int | None`	`None`	Override number of critical paths to include (defaults to `config.output.critical_path_report_`
`sprint_results`	`SprintPlanningResults | None`	`None`	Sprint planning results to embed (optional).
`project`	`Project | None`	`None`	Original project definition; enables richer output (optional).

Parameter	Type	Default	Description
`include_historic_base`	`bool`	`False`	Include historic baseline data when available.
`full_cost_detail`	`bool`	`False`	When `True`, includes per-task cost breakdowns and cost-distribution histograms in the JSON output.
`fx_provider`	`Any | None`	`None`	Foreign-exchange rate provider for multi-currency cost output (optional).
`target_budget`	`float | None`	`None`	Target budget for budget-confidence analysis in the output.
`target_hours`	`float | None`	`None`	Target duration (hours) for joint probability analysis in the output.

Example:

```
from mcprojsim.exporters import JSONExporter

JSONExporter.export(
    results,
    "results.json",
    config=config,
    project=project,
    critical_path_limit=5,
    sprint_results=sprint_results,
)
```

7.3 `CSVExporter`

Exports results as a CSV table format: metrics, percentiles, critical paths, histogram data, risk impact, resource diagnostics, and sprint data.

Method: `export(results, output_path, project=None, config=None, critical_path_limit`
`sprint_results=None, fx_provider=None) -> None`

Parameter	Type	Default	Description
`results`	`SimulationResults`	required	Simulation results object.
`output_path`	`Path | str`	required	File path for CSV output.
`project`	`Project | None`	`None`	Original project definition (optional; enables per-task detail).
`config`	`Config | None`	`None`	Active configuration; controls histogram bins and report limits.
`critical_path_limit`	`int | None`	`None`	Override number of critical paths to include.
`sprint_results`	`SprintPlanningResults | None`	`None`	Sprint planning results to embed (optional).
`fx_provider`	`Any | None`	`None`	Foreign-exchange rate provider for multi-currency cost output (optional).

Example:

```
from mcprojsim.exporters import CSVExporter

CSVExporter.export(results, "results.csv", config=config)
```

7.4 `HTMLExporter`

Exports a formatted, interactive HTML report with thermometers, percentile tables, charts (using matplotlib), critical-path analysis, staffing recommendations, and sprint-planning traceability.

Method: `export(results, output_path, project=None, config=None, critical_path_limit=No sprint_results=None, include_historic_base=False, fx_provider=None, target_budget=None, target_hours=None) -> None`

Parameter	Type	Default	Description
`results`	`SimulationResults`	required	Simulation results object.
`output_path`	`Path | str`	required	File path for HTML output.

Parameter	Type	Default	Description
`project`	`Project | None`	`None`	Original project definition; unlocks richer task and effort display (optional).
`config`	`Config | None`	`None`	Active configuration; used for T-shirt size labels and histogram bins (optional).
`critical_path_limit`	`int | None`	`None`	Override number of critical paths to include.
`sprint_results`	`SprintPlanningResults | None`	`None`	Sprint planning results to embed (optional).
`include_historic_base`	`bool`	`False`	Include historic baseline data when available.
`fx_provider`	`Any | None`	`None`	Foreign-exchange rate provider for multi-currency cost output (optional).
`target_budget`	`float | None`	`None`	Target budget for budget-confidence analysis in the output.
`target_hours`	`float | None`	`None`	Target duration (hours) for joint probability analysis in the output.

When `project` and `config` are provided:

- T-shirt-sized tasks are rendered with the active configuration labels.
- Story-point tasks show the configured hour ranges.
- Task descriptions and dependencies are included.
- Effort data is shown per task.

Example:

```
from mcprojsim.exporters import HTMLExporter

HTMLExporter.export(
    results,
    "report.html",
    project=project,
    config=config,
    critical_path_limit=10,
    sprint_results=sprint_results,
    include_historic_base=True
)
```

CHAPTER 8

Analysis Helpers

8.1 Overview

The `analysis` package provides five post-processing analyzers that extract deeper insights from a completed `SimulationResults` object. `StatisticalAnalyzer` computes descriptive statistics and confidence intervals; `SensitivityAnalyzer` ranks tasks by their Spearman rank correlation with total project duration; `CriticalPathAnalyzer` surfaces task criticality indices and the most frequent end-to-end critical sequences; `StaffingAnalyzer` recommends team sizes using a Brooks's Law–inspired communication-overhead model; and `CostAnalyzer` computes per-task cost sensitivity and the correlation between cost and schedule duration.

When to use this module: Use these analyzers when you need custom thresholds, deeper statistical breakdowns, or want to integrate simulation insights into your own tooling beyond what the CLI output provides.

Capability	Description
Descriptive statistics	Mean, median, std dev, CV, min/max on any duration array
Percentile calculation	Arbitrary percentile queries on simulation output
Confidence intervals	Bootstrap confidence interval at configurable confidence level
Sensitivity ranking	Spearman rank correlation of each task's duration against total project duration

Capability	Description
Critical-path analysis	Per-task criticality index (0–1) and most frequent full path sequences
Staffing recommendations	Team-size vs. delivery-date trade-offs across senior/mixed/junior profiles
Cost sensitivity analysis	Per-task Spearman rank correlation of task cost against total project cost, and Pearson correlation between total cost and total duration

Background: Spearman rank correlation — `SensitivityAnalyzer` converts each task's sampled durations and the total project durations to ranks before computing the correlation coefficient, making it robust to non-normal distributions and outliers. A coefficient near 1.0 means that task's duration strongly drives the overall schedule.

Background: Communication overhead — `StaffingAnalyzer` models individual productivity as `max(min_prod, 1 − c·(n−1))`, inspired by Brooks's Law, so adding team members beyond an optimal point yields diminishing and eventually negative returns on elapsed calendar time.

Imports:

```
from mcprojsim.analysis.statistics import StatisticalAnalyzer
from mcprojsim.analysis.sensitivity import SensitivityAnalyzer
from mcprojsim.analysis.critical_path import CriticalPathAnalyzer
from mcprojsim.analysis.staffing import StaffingAnalyzer
from mcprojsim.analysis.cost import CostAnalyzer, CostAnalysis
```

These specialized analysis modules provide focused statistical, sensitivity, and staffing analysis capabilities beyond what `SimulationResults` provides directly.

8.2 `StatisticalAnalyzer`

Convenience helpers for descriptive statistics on duration arrays.

Import:

```
from mcprojsim.analysis.statistics import StatisticalAnalyzer
```

Methods:

- **`calculate_statistics(durations: np.ndarray) -> dict[str, float]`** — Returns `mean`, `median`, `std_dev`, `variance`, `min`, `max`, `range`, `coefficient_of_variation`
- **`calculate_percentiles(durations: np.ndarray, percentiles: list[int]) ->`**

dict[int, float] — Compute specific percentiles

- **confidence_interval(durations: np.ndarray, confidence: float = 0.95) -> tuple[float, float]** — Return lower/upper bounds for a confidence interval

8.3 SensitivityAnalyzer

Analyzes which tasks have the strongest correlation with total project duration (Spearman rank correlation).

Import:

```
from mcprojsim.analysis.sensitivity import SensitivityAnalyzer
```

Methods:

- **calculate_correlations(results: SimulationResults) -> dict[str, float]** — Per-task correlation with total duration
- **get_top_contributors(results: SimulationResults, n: int = 10) -> list[tuple[str float]]** — Top N tasks by sensitivity

Example:

```
analyzer = SensitivityAnalyzer()
correlations = analyzer.calculate_correlations(results)

# Show top 5 sensitive tasks
top_5 = analyzer.get_top_contributors(results, n=5)
for task_id, correlation in top_5:
    print(f"{task_id}: correlation = {correlation:.3f}")
```

8.4 CriticalPathAnalyzer

Specialized analysis for critical paths and task criticality.

Import:

```
from mcprojsim.analysis.critical_path import CriticalPathAnalyzer
```

Methods:

- **get_criticality_index(results: SimulationResults) -> dict[str, float]** — Same as `results.get_critical_path()`
- **get_most_critical_tasks(results: SimulationResults, threshold: float = 0.5) -> list[str]** — Tasks appearing on critical path in >threshold of iterations
- **get_most_frequent_paths(results: SimulationResults, top_n: int | None = None) -> list[CriticalPathRecord]** — Most common paths (wraps `results.get_critic`

Example:

```
analyzer = CriticalPathAnalyzer()

# Tasks on critical path >80% of the time
critical_tasks = analyzer.get_most_critical_tasks(results, threshold=0.8)
print(f"Always-critical tasks: {critical_tasks}")

# Most common path
paths = analyzer.get_most_frequent_paths(results, top_n=1)
if paths:
    print(f"Most common path: {paths[0].format_path()}")
```

8.5 StaffingAnalyzer

Provides team-size recommendations and breaks down staffing by experience profile.

Import:

```
from mcprojsim.analysis.staffing import StaffingAnalyzer
```

Methods:

- **`individual_productivity(team_size: int, communication_overhead: float, min_producti float) -> float`** — Per-person productivity for a given team size. Returns a value in `[min_productivity, 1.0]`.
- **`effective_capacity(team_size: int, communication_overhead: float, productivity_fac float, min_productivity: float) -> float`** — Effective person-equivalents for a sized team, accounting for communication overhead and experience-level multiplier.
- **`calendar_hours(total_effort: float, critical_path_hours: float, team_size: int, communication_overhead: float, productivity_factor: float, min_productivity: float, hours_per_day: float) -> float`** — Elapsed calendar hours for a team. Bounded below by `critical_path_hours` regardless of team size.
- **`calculate_staffing_table(results: SimulationResults, config: Config) -> list[StaffingRow]`** — Full table of `StaffingRow` objects for every team size from 1 up to the observed peak parallelism, for each experience profile.
- **`recommend_team_size(results: SimulationResults, config: Config) -> list[StaffingRe`** — One optimal `StaffingRecommendation` per experience profile (the team size with the shortest calendar duration).

`StaffingRow` fields:

Field	Type	Description
`profile`	`str`	Experience level name
`team_size`	`int`	Candidate team size

Field	Type	Description
`individual_productivity`	`float`	Per-person productivity after communication overhead
`effective_capacity`	`float`	Total effective team capacity in person-equivalents
`calendar_hours`	`float`	Elapsed calendar hours needed
`calendar_working_days`	`int`	Calendar working days needed
`delivery_date`	`date | None`	Projected delivery date
`efficiency`	`float`	Calendar-time optimality: 1.0 means this team size achieves the minimum calendar duration for its profile

`StaffingRecommendation` fields:

Field	Type	Description
`profile`	`str`	Experience level name (e.g. `"senior"`, `"mixed"`, `"junior"`)
`recommended_team_size`	`int`	Optimal team size (minimum calendar duration)
`total_effort_hours`	`float`	Total effort basis in person-hours
`critical_path_hours`	`float`	Critical-path elapsed hours (lower bound on calendar time)
`calendar_working_days`	`int`	Calendar working days needed
`delivery_date`	`date | None`	Projected delivery date
`efficiency`	`float`	Always `1.0` — the recommended size is by definition the calendar-optimal team for its profile

Field	Type	Description
`parallelism_ratio`	`float`	Ratio of total effort to critical-path duration (degree of parallelism)
`effort_basis`	`str`	Effort basis label (`"mean"` or a percentile label such as `"P80"`)

Example:

```
analyzer = StaffingAnalyzer()

# Get full table for all profiles
table = analyzer.calculate_staffing_table(results, config)
for row in table:
    print(f"{row.profile} team (size {row.team_size}): "
          f"{row.calendar_working_days} days, efficiency {row.efficiency*100:.1f}%")

# Get primary recommendations
recommendations = analyzer.recommend_team_size(results, config)
for rec in recommendations:
    print(f"{rec.profile}: recommend {rec.recommended_team_size} people "
          f"({rec.efficiency*100:.0f}% efficiency)")
```

8.6 `CostAnalyzer`

Analyzes cost drivers and the relationship between cost and schedule duration. Only useful when the simulation produced cost data (`results.costs is not None`).

Import:

```
from mcprojsim.analysis.cost import CostAnalyzer, CostAnalysis
```

Methods:

- **`analyze(results: SimulationResults) -> CostAnalysis`** — Run cost sensitivity analysis and return a `CostAnalysis` data object. This is called automatically by the engine when cost data is present, and the result is stored in `results.cost_analysis`.

`CostAnalysis` fields (dataclass):

Field	Type	Description
`sensitivity`	`dict[str, float]`	Per-task Spearman rank correlation of that task's cost against total project cost. Higher absolute values indicate tasks whose cost variance most drives total cost variance.
`duration_correlation`	`float`	Pearson correlation between total project cost and total project duration. Values near 1.0 mean cost and schedule are tightly coupled; lower values suggest fixed costs or risks decouple them.

Example:

```
if results.cost_analysis:
    ca = results.cost_analysis
    print(f"Cost–duration correlation: {ca.duration_correlation:.3f}")

    # Top cost drivers
    top_drivers = sorted(ca.sensitivity.items(), key=lambda x: abs(x[1]), reverse=True)[:5]
    for task_id, corr in top_drivers:
        print(f"  {task_id}: cost sensitivity = {corr:.3f}")
```

CHAPTER 9

Validation Utilities

9.1 Overview

This module provides two lightweight utilities for integrating mcprojsim into scripts and pipelines. `Validator` offers a single static method that auto-detects a project file's format from its extension (`.yaml`/`.yml` or `.toml`), delegates to the appropriate parser, and returns a structured `(bool, str)` result — no parser knowledge required. `setup_logging` configures the named `mcprojsim` logger for library consumers, directing output to stdout with a timestamped format and a configurable log level.

When to use this module: Use `Validator` for pre-flight checks in CI/CD pipelines before running a simulation, and `setup_logging` when embedding mcprojsim as a library to control log verbosity without touching the root logger.

Capability	Description
Format auto-detection	Chooses `YAMLParser` or `TOMLParser` based on file extension
File existence check	Returns a clear error if the file path does not exist
Structured result	Returns `(True, "")` on success or `(False, error_message)` on failure
Logger configuration	Configures the `mcprojsim` named logger with a stdout handler and formatter
Configurable log level	Accepts `DEBUG`, `INFO`, `WARNING`, `ERROR`, or `CRITICAL` as a string

Imports:

```
from mcprojsim.utils import Validator
from mcprojsim.utils.logging import setup_logging
```

9.2 `Validator`

Convenience utility for validating project files by extension.

Import:

```
from mcprojsim.utils import Validator
```

Method:

- **`validate_file(file_path: Path | str) -> tuple[bool, str]`** — Returns `(True, "")` on success or `(False, error_message)` on failure. Selects the parser automatically based on file extension (`.yaml`/`.yml` or `.toml`). Returns an error for unsupported extensions or missing files.

Example:

```
is_valid, error = Validator.validate_file("project.yaml")
if not is_valid:
    print(f"Validation failed: {error}")
else:
    print("Project is valid!")
```

9.3 `setup_logging`

Configure logging verbosity and output for library usage.

Import:

```
from mcprojsim.utils import setup_logging
```

Method:

- **`setup_logging(level: str = "INFO") -> logging.Logger`** — Configure and return the `mcprojsim` named logger at the specified level. Accepted levels: `"DEBUG"`, `"INFO"`, `"WARNING"`, `"ERROR"`, `"CRITICAL"`.

Example:

```
logger = setup_logging(level="DEBUG")
logger.debug("Simulation started")
```

CHAPTER 10

Natural Language Parser

10.1 Overview

`NLProjectParser` converts semi-structured, informal plain text into valid mcprojsim YAML project files. It sits at the boundary between human-readable input and the structured project model, and is the backend for both the `mcprojsim generate` CLI command and the MCP server's `generate_project_file` tool. The parser processes input line by line using a section-based state machine: a header line (e.g. `Task 1:`, `Resource 2: Alice`) opens a new section, and subsequent bullet lines fill in that section's properties.

When to use this module: Use `NLProjectParser` when you want to create project files from natural language — for rapid prototyping, LLM-generated descriptions, or programmatic project creation without writing YAML by hand.

Capability	Description
`parse(text)`	Runs the state machine over plain text and returns a `ParsedProject` dataclass
`to_yaml(project)`	Renders a `ParsedProject` to a valid YAML project file string
`parse_and_generate(text)`	Convenience wrapper combining `parse` + `to_yaml` in one call
Flexible separators	Keyword–value separators `:`, `.`, `=`, or space are all equivalent
T-shirt size normalisation	Aliases like `Extra Large`, `2XL`, or `Medium` resolve to canonical sizes

Capability	Description
Resources & calendars	Parses `Resource N:` and `Calendar:` sections alongside tasks and sprint planning

Background: Section-based state machine — The parser reads input top to bottom. Any line matching a section-header pattern (`Task N:`, `Resource N:`, `Calendar:`, `Sprint planning:`, etc.) switches the current active section. Bullet lines (`-`, `*`, `•`) are dispatched to the active section's property handler. Project-level metadata (name, start date, hours per day) is matched anywhere outside a section.

Imports:

```
from mcprojsim.nl_parser import NLProjectParser
```

10.2 `NLProjectParser`

Converts semi-structured, plain-text project descriptions into valid mcprojsim YAML project files. Also available via the `mcprojsim generate` CLI command and the MCP server's `generate_project_file` tool.

```
from mcprojsim.nl_parser import NLProjectParser

parser = NLProjectParser()
```

Method	Signature	Description
`parse`	`(text: str) -> ParsedProject`	Extract project metadata and tasks from a text description. Returns a `ParsedProject`. Raises `ValueError` if no tasks are found.
`to_yaml`	`(project: ParsedProject) -> str`	Render a `ParsedProject` as a valid YAML project file string.
`parse_and_generate`	`(text: str) -> str`	Convenience wrapper: calls `parse` then `to_yaml` and returns the YAML string directly.

10.2.1 Input format

The parser processes the description line by line using a section-based state machine. A **section header** line (e.g. `Task 1:`, `Resource 2: Alice`, `Sprint planning:`) opens a new section; subsequent **bullet lines** (prefixed with `-`, `*`, or `•`) are parsed as properties of that section. Blank lines are ignored. Project-level metadata (name, start date, etc.) can appear anywhere outside a section.

Separators are flexible. In most patterns the separator between keyword and value can be `:`, `.`, `=`, or a space, and the keyword itself is case-insensitive. For example, all of the following are equivalent:

```
Size: M
Size. M
Size = M
size XL
```

10.2.2 Project-level metadata

These lines can appear before the first task or between sections:

Keyword	Example	Notes
`Project name:` / `Project:`	`Project name: Website Redesign`	
`Start date:`	`Start date: 2026-06-01`	ISO 8601 (`YYYY-MM-DD`)
`Description:`	`Description: Q3 infrastructure work`	
`Hours per day:`	`Hours per day: 7.5`	Default `8.0`
`Confidence levels:`	`Confidence levels: 50, 80, 90, 95`	Comma-separated percentiles
`Rate:` / `Hourly rate:` / `Default rate:` / `Blended rate:`	`Rate: $150/hour`	Default hourly rate (cost estimation)
`Overhead:` / `Overhead rate:`	`Overhead: 15%`	Overhead multiplier applied to all costs
`Currency:`	`Currency: USD`	Three-letter ISO 4217 currency code

10.2.3 Tasks

A task section starts with `Task N:` optionally followed by the task name on the same line. Subsequent bullet lines define properties.

Alternatively, when no `Task N:` headers are present, the parser auto-detects tasks from plain lists:

List format	Example	Numbering
Dot numbered	`1. Design phase`	Preserved from source
Paren numbered	`2) Implementation`	Preserved from source
Bracket numbered	`[3] Testing`	Preserved from source
Bullet (-, *, •)	`- Backend API`	Auto-assigned (1, 2, 3, …)
Hash numbered	`# 1 First task`	Preserved from source

Auto-task mode activates only when **no** `Task N:` headers have been seen. The two modes cannot be mixed.

Indented continuation lines under auto-detected tasks are parsed as bullet properties, identical to the explicit-header format.

Inline properties on task lines

When using auto-detected lists, properties can appear inline on the task name line:

Inline pattern	Example	Effect
Bracketed size	`Backend API [XL]`	Sets `t_shirt_size = "XL"`
Parenthesized size	`Frontend (M)`	Sets `t_shirt_size = "M"`
Fuzzy size hint	`probably an M`, `likely L`, `assume S`	Sets `t_shirt_size`
Inline range	`3–5 days`, `2-4 hours`	Sets `low`, `expected`, `high`, `unit`
Inline dependency	`depends on Task 1`	Sets `dependency_refs`

The parser extracts inline properties and strips them from the task name.

Task bullet properties

Bullet keyword	Example	Notes
`Name:`	`- Name: Backend API`	Also accepted as first unmatched bullet
`Size:`	`- Size: M`	See size aliases below
`Story points:` / `Points:`	`- Story points: 5`	
`Estimate:`	`- Estimate: 3/5/10 days`	`low/expected/high`, separator `/` - or `,`
`Depends on:` / `Depends:` / `Depend on:`	`- Depends on Task 1, Task 3`	References by task number
`Resources:`	`- Resources: Alice, Bob`	Names must match a `Resource N:` header
`Max resources:`	`- Max resources: 2`	Concurrent resource cap
`Min experience:` / `Min experience level:`	`- Min experience: 2`	1–3
`Fixed cost:` / `One-time cost:`	`- Fixed cost: 5000`	One-time cost added regardless of duration
`Risk:` / `Risks:`	`- Risk: Integration failure`	Opens a risk sub-section under this task (see below)

Risk bullet properties (nested under a `Risk:` line within a task section):

Bullet keyword	Example	Notes
`Probability:`	`- Probability: 20%`	0–100
`Impact:`	`- Impact: 5 days`	Schedule impact; optional unit
`Cost impact:`	`- Cost impact: $10000`	Monetary impact if the risk occurs

Risks can also be described in **prose form** inside a task: *"there is a 15% chance*

of a 3-day delay" or *"a 10% risk of $5000 penalty"*.

Estimate units for explicit estimates: `hours` / `hour` / `h`, `days` / `day` / `d`, `weeks` / `week` / `w`.

T-shirt size aliases — all of the following map to a canonical size:

Canonical	Accepted aliases
`XS`	`XS`, `Extra Small`, `Extrasmall`
`S`	`S`, `Small`
`M`	`M`, `Medium`, `Med`
`L`	`L`, `Large`
`XL`	`XL`, `Extra Large`, `Extralarge`
`XXL`	`XXL`, `Extra Extra Large`, `2XL`

10.2.4 Resources

A resource section starts with `Resource N: Name`. Bullet properties:

Bullet keyword	Example	Notes
`Experience level:` / `Experience:`	`- Experience: 3`	1–3
`Productivity level:` / `Productivity:`	`- Productivity: 1.1`	Multiplier, default `1.0`
`Availability:`	`- Availability: 0.8`	Fraction of full-time, 0–1
`Calendar:`	`- Calendar: part_time`	References a `Calendar:` section ID
`Sickness prob:` / `Sickness:`	`- Sickness: 0.02`	Per-day probability
`Rate:` / `Hourly rate:` / `Cost:`	`- Rate: $120/hour`	Per-resource hourly rate (cost estimation)
`Absence:` / `Planned absence:`	`- Absence: 2026-05-15, 2026-06-01`	Comma-separated ISO dates or date ranges (`2026-05-20 to 2026-05-22`)

10.2.5 Calendars

A calendar section starts with `Calendar: id`. Bullet properties:

Bullet keyword	Example	Notes
`Work hours per day:` / `Work hours:`	`- Work hours: 7`	
`Work days:`	`- Work days: 1, 2, 3, 4`	Integers, 1=M
`Holidays:`	`- Holidays: 2026-12-25, 2026-12-26`	ISO 8601 date

10.2.6 Sprint planning

A `Sprint planning:` header opens the sprint planning section. Bullet properties:

Bullet keyword	Example	Notes
`Sprint length:`	`- Sprint length: 2`	Weeks; also `2-week sprints`
`Capacity mode:`	`- Capacity mode: story points`	`story points` or `tasks`
`Planning confidence level:`	`- Planning confidence level: 80%`	
`Velocity model:`	`- Velocity model: empirical`	`empirical` or `neg_binomial`
`Removed work treatment:`	`- Removed work treatment: churn_only`	`churn_only` or `reduce_backlog`
`Sickness:`	`- Sickness: enabled`	`enabled`/`disabled`/`on`/`off`/ also `No sickness`
`Sickness team size:`	`- Sickness team size: 6`	
`Sickness probability per person per week:`	`- Sickness probability: 5%`	
`Sickness duration log mu:`	`- Sickness duration log mu: 1.1`	
`Sickness duration log sigma:`	`- Sickness duration log sigma: 0.4`	

Sprint history entries use `Sprint history <id>:` as the header (auto-ID generated as `SPR-001`, `SPR-002`, … if omitted). Bullet keywords:

Bullet keyword	Aliases	Notes
`Completed:`	`Done:`, `Finished:`, `Delivered:`	`10 points` or `10 tasks`
`Spillover:`	`Carryover:`, `Rolled over:`	
`Added:`	`Scope added:`	
`Removed:`	`Scope removed:`	
`Holiday factor:`		Capacity reduction factor

Future sprint overrides use `Future sprint override <N>:` or `Future sprint override <YYYY-MM-DD>:` as the header. Bullet properties: `Sprint number:`, `Start date:`, `Holiday factor:`, `Capacity multiplier:`, `Notes:`.

10.2.7 Complete example

```
Project name: Platform Migration
Start date: 2026-05-01

Resource 1: Alice
- Experience: 3
- Productivity: 1.1
- Sickness: 0.02
- Absence: 2026-05-15

Resource 2: Bob
- Experience: 2
- Availability: 0.8

Calendar: default
- Work hours: 8
- Work days: 1, 2, 3, 4, 5
- Holidays: 2026-05-25

Task 1: Architecture design
- Estimate: 16/24/40 hours
- Min experience: 2

Task 2: Core implementation
- Estimate: 80/120/180 hours
- Depends on Task 1
- Resources: Alice, Bob
- Max resources: 2
- Min experience: 2
```

```
from mcprojsim.nl_parser import NLProjectParser

yaml_output = NLProjectParser().parse_and_generate(description)
```

10.3 Data Classes

10.3.1 `ParsedProject`

Top-level container for all data extracted from a natural language description.

Field	Type	Default	Description
`name`	`str`	`"Untitled Project"`	Project name.
`start_date`	`str | None`	`None`	ISO 8601 start date string.

Field	Type	Default	Description
`description`	`str | None`	`None`	Optional project description.
`hours_per_day`	`float`	`8.0`	Working hours per day.
`tasks`	`list[ParsedTask]`	`[]`	Extracted tasks.
`confidence_levels`	`list[int]`	`[50, 80, 90, 95]`	Percentile confidence levels to report.
`resources`	`list[ParsedResource]`	`[]`	Extracted team members/resources.
`calendars`	`list[ParsedCalendar]`	`[]`	Extracted working calendars.
`sprint_planning`	`ParsedSprintPlanning | None`	`None`	Sprint planning configuration, if present.
`default_hourly_rate`	`float | None`	`None`	Default hourly rate for cost estimation.
`overhead_rate`	`float | None`	`None`	Overhead multiplier applied to all costs.
`currency`	`str | None`	`None`	Three-letter ISO 4217 currency code.

10.3.2 `ParsedTask`

A single task extracted from the description.

Field	Type	Default	Description
`number`	`int`	required	Task number as written in the input (e.g., `1` for `Task 1:`).
`name`	`str`	`""`	Task name.
`t_shirt_size`	`str | None`	`None`	Normalised T-shirt size label (e.g., `"M"`, `"XL"`).
`story_points`	`int | None`	`None`	Story point estimate.
`low_estimate`	`float | None`	`None`	Optimistic explicit estimate.
`expected_estimate`	`float | None`	`None`	Expected explicit estimate.
`high_estimate`	`float | None`	`None`	Pessimistic explicit estimate.
`estimate_unit`	`str`	`"days"`	Unit for explicit estimates (`"days"` or `"hours"`).

Field	Type	Default	Description
`dependency_refs`	`list[str]`	`[]`	Raw dependency references as written (e.g., `["Task 1"]`).
`description`	`str | None`	`None`	Optional task description text.
`resources`	`list[str]`	`[]`	Resource names assigned to the task.
`max_resources`	`int`	`1`	Maximum number of resources that can work the task concurrently.
`min_experience_level`	`int`	`1`	Minimum experience level required for an assigned resource.
`fixed_cost`	`float | None`	`None`	One-time fixed cost added to this task regardless of duration.
`risks`	`list[ParsedRisk]`	`[]`	Risks associated with this task.

10.3.3 `ParsedResource`

A team member or resource extracted from the description.

Field	Type	Default	Description
`number`	`int`	required	Resource number as written in the input (e.g., `1` for `Resource 1:`).
`name`	`str`	`""`	Resource name.
`availability`	`float`	`1.0`	Fraction of working time available (0.0–1.0).
`experience_level`	`int`	`2`	Experience level (1–3; 1 = junior, 2 = mid-level, 3 = senior).
`productivity_level`	`float`	`1.0`	Productivity multiplier.
`calendar`	`str`	`"default"`	Calendar ID used by this resource.
`sickness_prob`	`float`	`0.0`	Per-day sickness probability.

Field	Type	Default	Description
`planned_absence`	`list[str]`	`[]`	List of planned absence date strings.
`hourly_rate`	`float | None`	`None`	Per-resource hourly rate for cost estimation.

10.3.4 `ParsedRisk`

A risk associated with a task, extracted from the description.

Field	Type	Default	Description
`name`	`str`	`""`	Risk name.
`probability`	`float`	`0.0`	Probability of occurrence (0.0–1.0 or percentage divided by 100).
`impact_value`	`float | None`	`None`	Schedule impact amount.
`impact_unit`	`str | None`	`None`	Unit for the schedule impact (`"days"`, `"hours"`, etc.).
`cost_impact`	`float | None`	`None`	Monetary impact if the risk occurs.

10.3.5 `ParsedCalendar`

A working calendar extracted from the description.

Field	Type	Default	Description
`id`	`str`	`"default"`	Calendar identifier.
`work_hours_per_day`	`float`	`8.0`	Working hours per day.
`work_days`	`list[int]`	`[1, 2, 3, 4, 5]`	Working days of the week (1 = Monday … 7 = Sunday).
`holidays`	`list[str]`	`[]`	ISO 8601 holiday date strings.

10.3.6 `ParsedSprintPlanning`

Sprint planning configuration extracted from the description.

Field	Type	Default	Description
`enabled`	`bool`	`True`	Whether sprint planning is enabled.
`sprint_length_weeks`	`int`	`2`	Sprint length in weeks.
`capacity_mode`	`str`	`"story_points"`	Capacity tracking mode (`"story_points"` or `"tasks"`).
`planning_confidence_level`	`float | None`	`None`	Confidence level override for commitment guidance.
`removed_work_treatment`	`str | None`	`None`	How removed work is handled (`"churn_only"` or `"reduce_backlog"`).
`velocity_model`	`str | None`	`None`	Velocity model override (`"empirical"` or `"neg_binomial"`).
`sickness_enabled`	`bool | None`	`None`	Override for sickness simulation.
`sickness_team_size`	`int | None`	`None`	Team size for sickness modelling.
`sickness_probability_per_person_per_week`	`float | None`	`None`	Per-person-per-week sickness probability override.
`sickness_duration_log_mu`	`float | None`	`None`	Log-mean for sickness duration distribution override.
`sickness_duration_log_sigma`	`float | None`	`None`	Log-sigma for sickness duration distribution override.
`future_sprint_overrides`	`list[ParsedFutureSprintOverride]`	`[]`	Capacity overrides for future sprints.
`history`	`list[ParsedSprintHistoryEntry]`	`[]`	Historical sprint data.

10.3.7 `ParsedFutureSprintOverride`

A capacity override for a future sprint.

Field	Type	Default	Description
`sprint_number`	`int | None`	`None`	Sprint number this override applies to.

Field	Type	Default	Description
`start_date`	`str | None`	`None`	ISO 8601 start date for the overridden sprint.
`holiday_factor`	`float | None`	`None`	Capacity reduction factor due to holidays (0.0–1.0).
`capacity_multiplier`	`float | None`	`None`	Overall capacity multiplier for the sprint.
`notes`	`str | None`	`None`	Free-text notes for this override.

10.3.8 `ParsedSprintHistoryEntry`

A historical sprint outcome.

Field	Type	Default	Description
`sprint_id`	`str`	required	Unique sprint identifier.
`completed_story_points`	`float | None`	`None`	Story points completed in the sprint.
`completed_tasks`	`int | None`	`None`	Tasks completed (task-capacity mode).
`spillover_story_points`	`float | None`	`None`	Story points that spilled over to the next sprint.
`spillover_tasks`	`int | None`	`None`	Tasks that spilled over (task-capacity mode).
`added_story_points`	`float | None`	`None`	Story points added mid-sprint.
`added_tasks`	`int | None`	`None`	Tasks added mid-sprint (task-capacity mode).
`removed_story_points`	`float | None`	`None`	Story points removed mid-sprint.
`removed_tasks`	`int | None`	`None`	Tasks removed mid-sprint (task-capacity mode).
`holiday_factor`	`float | None`	`None`	Capacity reduction factor applied to this sprint.

CHAPTER 11

API Examples

11.1 Overview

This page contains complete, runnable code examples that show end-to-end usage of the mcprojsim public API. Each example is self-contained and covers a distinct integration pattern — from single-project simulation to portfolio batch processing and web dashboard embedding. They are designed to serve as integration test templates or as starting points for custom tooling built on top of mcprojsim.

For parameter details, consult the API reference pages for `SimulationEngine`, `Config`, `YAMLParser`, the analysis modules, and exporters.

Example	What it demonstrates
1 — Programmatic Simulation with Full Analysis	Load a YAML project, run a simulation, run critical-path / sensitivity / staffing analysis, export JSON and HTML
2 — Resource-Constrained and Two-Pass Scheduling	Build a `Project` model in code, enable two-pass scheduling, inspect resource utilisation metrics
3 — Dashboard Integration	Embed mcprojsim in a Flask API endpoint; parse a project file and return percentile results as JSON
4 — Sprint Planning with Forecast	Run `SprintSimulationEngine` on a sprint-enabled project and build a sprint-by-sprint delivery forecast

Example	What it demonstrates
5 — Batch Processing Multiple Projects	Scan a directory of YAML files, simulate each project, and aggregate results into a pandas DataFrame
6 — Configuration-Driven Customization	Load a `Config` from file and apply programmatic overrides before running a simulation
7 — Progress Callback and Cancellation	Use `progress_callback` to receive live progress updates and `cancel()` to abort a running simulation from another thread
8 — Cost Estimation and Budget Analysis	Enable cost tracking, query budget confidence, and identify cost drivers

---

11.2 Example 1: Programmatic Simulation with Full Analysis

This example shows how an external tool can load a project, run a simulation, and perform comprehensive analysis:

```
from mcprojsim import SimulationEngine
from mcprojsim.config import Config
from mcprojsim.parsers import YAMLParser
from mcprojsim.analysis.critical_path import CriticalPathAnalyzer
from mcprojsim.analysis.staffing import StaffingAnalyzer
from mcprojsim.analysis.sensitivity import SensitivityAnalyzer
from mcprojsim.exporters import HTMLExporter, JSONExporter

def run_comprehensive_analysis(project_file: str, output_dir: str):
    """Load, simulate, analyze, and export a complete project report."""

    # Step 1: Load project and configuration
    project = YAMLParser().parse_file(project_file)
    config = Config.get_default()

    # Step 2: Run simulation
    engine = SimulationEngine(
        iterations=10000,
        random_seed=42,
        config=config,
        show_progress=True
    )
    results = engine.run(project)

    # Step 3: Critical path analysis
    cp_analyzer = CriticalPathAnalyzer()
    critical_tasks = cp_analyzer.get_most_critical_tasks(results, threshold=0.7)
    print(f"Critical tasks (>70% frequency): {critical_tasks}")

    # Step 4: Sensitivity analysis
    sens_analyzer = SensitivityAnalyzer()
    top_risks = sens_analyzer.get_top_contributors(results, n=5)
    print("\nTop 5 sensitivity contributors:")
    for task_id, correlation in top_risks:
        print(f"  {task_id}: {correlation:.3f}")

    # Step 5: Staffing recommendations
    staff_analyzer = StaffingAnalyzer()
    recommendations = staff_analyzer.recommend_team_size(results, config)
    for rec in recommendations:
        print(f"Staffing ({rec.profile}): {rec.recommended_team_size} people, "
              f"{rec.calendar_working_days} days, {rec.efficiency*100:.0f}% efficiency")

    # Step 6: Export all formats
    base_path = f"{output_dir}/analysis"
    JSONExporter.export(results, f"{base_path}.json", config=config, project=project)
    HTMLExporter.export(results, f"{base_path}.html", project=project, config=config)

    return results

# Usage
```

11.3 Example 2: Resource-Constrained and Two-Pass Scheduling

This example shows how to enable resource-constrained scheduling and two-pass mode:

```
from datetime import date
from mcprojsim import SimulationEngine
from mcprojsim.config import Config
from mcprojsim.models.project import (
    Project, ProjectMetadata, Task, TaskEstimate,
    ResourceSpec, CalendarSpec,
)

# Define resources and calendars
project = Project(
    project=ProjectMetadata(
        name="Constrained Project",
        start_date=date(2026, 6, 1),
    ),
    tasks=[
        Task(
            id="task_a",
            name="Backend work",
            estimate=TaskEstimate(low=20, expected=40, high=80),
            resources=["dev_team"],
        ),
        Task(
            id="task_b",
            name="Frontend work",
            estimate=TaskEstimate(low=16, expected=32, high=64),
            resources=["dev_team"],
        ),
        Task(
            id="task_c",
            name="Integration",
            estimate=TaskEstimate(low=8, expected=16, high=32),
            dependencies=["task_a", "task_b"],
            resources=["dev_team"],
        ),
    ],
    resources=[
        ResourceSpec(name="dev_team", availability=1.0, experience_level=3),
        ResourceSpec(name="dev_team", availability=0.8, experience_level=2),
    ],
    calendars=[
        CalendarSpec(
            id="default",
            work_hours_per_day=8.0,
            work_days=[1, 2, 3, 4, 5],
            holidays=[date(2026, 7, 4)],
        ),
    ],
)

config = Config.get_default()
```

11.4 Example 3: Dashboard Integration

This example demonstrates integrating mcprojsim into a web dashboard:

```
from flask import Flask, request, jsonify
from mcprojsim import SimulationEngine
from mcprojsim.config import Config
from mcprojsim.models.project import Project
from pathlib import Path
import json

app = Flask(__name__)

@app.route('/api/simulate', methods=['POST'])
def simulate_project():
    """API endpoint for running simulations."""

    # Parse request
    data = request.json
    project_file = data.get('project_file')
    iterations = data.get('iterations', 5000)

    # Load and validate
    from mcprojsim.parsers import YAMLParser
    try:
        project = YAMLParser().parse_file(project_file)
    except Exception as e:
        return jsonify({'error': str(e)}), 400

    # Run simulation
    config = Config.get_default()
    engine = SimulationEngine(iterations=iterations, config=config)
    results = engine.run(project)

    # Build response with key metrics
    return jsonify({
        'project_name': results.project_name,
        'iterations': results.iterations,
        'mean_hours': float(results.mean),
        'mean_days': float(results.mean / results.hours_per_day),
        'percentiles': {
            'p50': float(results.percentile(50)),
            'p80': float(results.percentile(80)),
            'p95': float(results.percentile(95)),
        },
        'critical_tasks': list(results.get_critical_path().keys())[:10],
        'success_at_500h': float(results.probability_of_completion(500)),
    })

@app.route('/api/config', methods=['GET'])
def get_config():
    """Return active configuration."""
    config = Config.get_default()
    return jsonify({
        't_shirt_categories': config.get_t_shirt_categories(),
```

11.5 Example 4: Sprint Planning with Forecast

This example shows sprint-planning integration:

```
from mcprojsim.parsers import YAMLParser
from mcprojsim.planning.sprint_engine import SprintSimulationEngine

def forecast_project_completion(project_file: str) -> dict:
    """Forecast sprint-by-sprint delivery timeline."""

    project = YAMLParser().parse_file(project_file)

    # Sprint planning must be enabled
    if not project.sprint_planning or not project.sprint_planning.enabled:
        raise ValueError("Sprint planning not enabled in project")

    # Run sprint simulation
    engine = SprintSimulationEngine(iterations=5000, random_seed=42)
    results = engine.run(project)

    # Build forecast
    forecast = {
        'project_name': results.project_name,
        'sprint_length_weeks': results.sprint_length_weeks,
        'forecasts': {
            'p50_sprints': results.percentile(50),
            'p80_sprints': results.percentile(80),
            'p50_date': results.date_percentile(50),
            'p80_date': results.date_percentile(80),
        },
        'guidance': {
            'recommended_capacity': results.planned_commitment_guidance,
            'historical_velocity': results.historical_diagnostics.get('series_statistics', {}).g
            'risk_level': 'low' if results.std_dev < results.mean * 0.2 else 'high',
        }
    }

    return forecast

# Usage
forecast = forecast_project_completion("sprint_project.yaml")
print(f"P80 completion: {forecast['forecasts']['p80_date']}")
```

11.6 Example 5: Batch Processing Multiple Projects

This example processes multiple projects and generates a portfolio view:

```
from pathlib import Path
from mcprojsim import SimulationEngine
from mcprojsim.config import Config
from mcprojsim.parsers import YAMLParser
import pandas as pd

def analyze_portfolio(project_dir: str) -> pd.DataFrame:
    """Run simulations for all projects in a directory and create summary."""

    results_list = []
    config = Config.get_default()

    for project_file in Path(project_dir).glob("*.yaml"):
        print(f"Processing {project_file.name}...")

        try:
            project = YAMLParser().parse_file(str(project_file))
            engine = SimulationEngine(iterations=5000, config=config, show_progress=False)
            results = engine.run(project)

            results_list.append({
                'project': project.project.name,
                'tasks': len(project.tasks),
                'mean_hours': results.mean,
                'mean_days': results.mean / results.hours_per_day,
                'p80_hours': results.percentile(80),
                'p95_hours': results.percentile(95),
                'risk_high': len([t for t in results.get_critical_path().items()
                                  if t[1] > 0.8]),
            })
        except Exception as e:
            print(f"  Error: {e}")

    # Create DataFrame
    df = pd.DataFrame(results_list)
    df = df.sort_values('mean_days', ascending=False)

    print("\n=== Portfolio Summary ===")
    print(df.to_string(index=False))
    print(f"\nTotal projected effort: {df['mean_hours'].sum():.0f} hours")

    return df

# Usage
portfolio = analyze_portfolio("projects/")
```

11.7 Example 6: Configuration-Driven Customization

This example shows how to customize the simulation via configuration:

```
from mcprojsim.config import Config
from mcprojsim import SimulationEngine
from mcprojsim.parsers import YAMLParser
import yaml

def simulate_with_custom_config(project_file: str, config_file: str, overrides: dict):
    """Run simulation with config file + programmatic overrides."""

    # Load config from file
    config = Config.load_from_file(config_file)

    # Apply programmatic overrides
    if 'number_bins' in overrides:
        config.output.number_bins = overrides['number_bins']

    if 'iterations' in overrides:
        iterations = overrides['iterations']
    else:
        iterations = config.simulation.default_iterations

    if 'staffing_percentile' in overrides:
        config.staffing.effort_percentile = overrides['staffing_percentile']

    # Run simulation with customized config
    project = YAMLParser().parse_file(project_file)
    engine = SimulationEngine(
        iterations=iterations,
        random_seed=42,
        config=config
    )
    results = engine.run(project)

    return results

# Usage - override histogram bins to 100 and use P80 for staffing
results = simulate_with_custom_config(
    'project.yaml',
    'config.yaml',
    {
        'number_bins': 100,
        'staffing_percentile': 80,
        'iterations': 20000,
    }
)
```

11.8 Example 7: Progress Callback and Cancellation

This example demonstrates two features added to `SimulationEngine`:

1. **`progress_callback`** — receive live `(completed, total)` updates during a simulation instead of progress being printed to stdout.
2. **`cancel()`** — abort a running simulation from any thread, causing `run()` to raise `SimulationCancelled`.

11.8.1 7a — Progress callback (basic usage)

```
from datetime import date
from mcprojsim import SimulationEngine
from mcprojsim.models.project import Project, ProjectMetadata, Task, TaskEstimate


def on_progress(completed: int, total: int) -> None:
    pct = 100 * completed / total
    print(f"\r  Simulating… {pct:5.1f}%  ({completed}/{total})", end="", flush=True)


project = Project(
    project=ProjectMetadata(name="Callback Demo", start_date=date(2026, 5, 1)),
    tasks=[
        Task(id="t1", name="Design",  estimate=TaskEstimate(low=2, expected=5, high=10)),
        Task(id="t2", name="Build",   estimate=TaskEstimate(low=5, expected=10, high=20),
             dependencies=["t1"]),
        Task(id="t3", name="Test",    estimate=TaskEstimate(low=3, expected=6, high=12),
             dependencies=["t2"]),
    ],
)

engine = SimulationEngine(
    iterations=10000,
    random_seed=42,
    show_progress=True,          # would normally print to stdout …
    progress_callback=on_progress,  # … but the callback takes over instead
)
results = engine.run(project)
print(f"\nDone — P80: {results.percentile(80):.1f} hours")
```

When `progress_callback` is set, all stdout progress output is suppressed. The callback receives `(completed_iterations, total_iterations)` updates from the engine even when `show_progress=False`. In single-pass mode the total is just `iterations`. In two-pass mode the total is `pass1_iterations + iterations`, so progress keeps moving forward across both phases. Sequential runs report at the built-in 10 %

cadence, while parallel runs report as chunks complete, so update frequency can be higher on larger worker counts.

!!! note `show_progress` only controls stdout progress rendering. It does not disable the callback itself.

11.8.2 7b — Cancellation from another thread

```
import threading
from datetime import date
from mcprojsim import SimulationEngine
from mcprojsim.simulation.engine import SimulationCancelled
from mcprojsim.models.project import Project, ProjectMetadata, Task, TaskEstimate

project = Project(
    project=ProjectMetadata(name="Cancel Demo", start_date=date(2026, 5, 1)),
    tasks=[
        Task(id="t1", name="Design",  estimate=TaskEstimate(low=2, expected=5, high=10)),
        Task(id="t2", name="Build",   estimate=TaskEstimate(low=5, expected=10, high=20),
             dependencies=["t1"]),
        Task(id="t3", name="Test",    estimate=TaskEstimate(low=3, expected=6, high=12),
             dependencies=["t2"]),
    ],
)

engine = SimulationEngine(iterations=500_000, random_seed=42, show_progress=False)


def run_in_background():
    try:
        results = engine.run(project)
        print(f"Finished: mean = {results.mean:.1f} hours")
    except SimulationCancelled:
        print("Simulation was cancelled.")


worker = threading.Thread(target=run_in_background)
worker.start()

# Cancel after a brief delay (e.g. user clicks "Stop")
import time
time.sleep(0.1)
engine.cancel()

worker.join(timeout=5.0)
assert not worker.is_alive(), "Worker should have stopped"
```

The `cancel()` method sets an internal flag that is checked at the top of every iteration. Because the check happens per-iteration, the engine finishes the current iteration before raising `SimulationCancelled`.

11.8.3 7c — Combining callback with cancellation (GUI pattern)

A common GUI integration pattern is to use the callback for progress bar updates and cancel from a UI button:

```
import threading
from datetime import date
from mcprojsim import SimulationEngine
from mcprojsim.simulation.engine import SimulationCancelled
from mcprojsim.models.project import Project, ProjectMetadata, Task, TaskEstimate

project = Project(
    project=ProjectMetadata(name="GUI Pattern", start_date=date(2026, 5, 1)),
    tasks=[
        Task(id="t1", name="Design",  estimate=TaskEstimate(low=2, expected=5, high=10)),
        Task(id="t2", name="Build",   estimate=TaskEstimate(low=5, expected=10, high=20),
             dependencies=["t1"]),
        Task(id="t3", name="Test",    estimate=TaskEstimate(low=3, expected=6, high=12),
             dependencies=["t2"]),
    ],
)


# --- Simulate a GUI progress bar ---
progress_pct = 0


def update_progress_bar(completed: int, total: int) -> None:
    global progress_pct
    progress_pct = int(100 * completed / total)


engine = SimulationEngine(
    iterations=200_000,
    random_seed=42,
    show_progress=True,
    progress_callback=update_progress_bar,
)


def worker_fn():
    try:
        results = engine.run(project)
        print(f"Done — mean {results.mean:.1f} h, P80 {results.percentile(80):.1f} h")
    except SimulationCancelled:
        print(f"Cancelled at {progress_pct}%")


worker = threading.Thread(target=worker_fn)
worker.start()

# Simulate the user pressing "Cancel" after a short delay
import time
time.sleep(0.05)
engine.cancel()
```

11.9 Example 8 — Cost Estimation and Budget Analysis

Enable cost tracking by setting `default_hourly_rate` on the project metadata. The engine computes per-iteration costs (effort × rate + fixed costs + risk cost impacts), and `SimulationResults` exposes budget-analysis helpers.

```
from datetime import date
from mcprojsim.models.project import (
    Project, ProjectMetadata, Task, TaskEstimate, Risk, RiskImpact, ResourceSpec,
)
from mcprojsim.simulation.engine import SimulationEngine

project = Project(
    project=ProjectMetadata(
        name="Payment Gateway",
        start_date=date(2026, 6, 1),
        default_hourly_rate=120.0,
        overhead_rate=0.15,
        currency="USD",
    ),
    tasks=[
        Task(
            id="design",
            name="API Design",
            estimate=TaskEstimate(low=16, expected=24, high=40),
            fixed_cost=2000.0,  # licensing fee
        ),
        Task(
            id="impl",
            name="Implementation",
            estimate=TaskEstimate(low=80, expected=120, high=200),
            dependencies=["design"],
            risks=[
                Risk(
                    id="impl_vendor",
                    name="Vendor API instability",
                    probability=0.25,
                    impact=RiskImpact(type="absolute", value=40, unit="hours"),
                    cost_impact=8000.0,
                ),
            ],
        ),
        Task(
            id="qa",
            name="QA & Certification",
            estimate=TaskEstimate(low=40, expected=60, high=100),
            dependencies=["impl"],
        ),
    ],
    resources=[
        ResourceSpec(name="Alice", hourly_rate=150.0),
        ResourceSpec(name="Bob"),  # uses default_hourly_rate
    ],
)

engine = SimulationEngine(iterations=10_000, random_seed=42)
results = engine.run(project)
```